Blind Grading Packet

For the second grader of "Why Compressed LLM Memory Forgets"

Instructions

Thank you for helping. This packet has two parts and takes about two to three hours in total.

Part A (about 2 hours). For each of the 15 studies, read our reading notes (and, if you wish, the paper at the link) and assign one grade to each of the four terms: Capacity, Addressing, Interference and Write distortion. Use only the rules in Section 2 of this packet. Grades: M (measured), B (bounded), I (inferred), U (unassigned: the evidence mixes terms), or – (the study does not address this term). Remember two caps: Capacity can only be B, U or –; a study marked "read in abstract only" can get at most I (U for Capacity).

Part B (about 30 minutes). For each of the 40 papers, read the title and abstract and answer: is it relevant to a survey of information loss in compressed memory of LLM systems (KV caches, prompt compression, retrieval, agent memory, or associative recall in sequence models)? Answer yes or no.

Please work alone. Do not look at the paper's Table 1, the author's grades, or any earlier spot-check sheet, and do not discuss items with the author until you have finished. Write your answers in `answer_sheet.csv` (or on paper, one line per item). Use the note column for anything you were unsure about. There are no trick questions; disagreement is useful information.

1. Plain-language glossary

LLM large language model, e.g. the model behind a chatbot.

Context the text the model reads before answering (documents, chat history, the question).

KV cache the model's working memory of the context: one stored entry (a key and a value) per token.

Token a word piece; the unit the model reads. A page of text is roughly 500 tokens.

Eviction deleting cache entries to save memory. SnapKV, H2O, TOVA and StreamingLLM are eviction policies.

Quantization storing numbers with fewer bits (e.g. 2 instead of 16), which saves memory but adds rounding error.

Prompt compression shortening the input text or replacing it with a few learned vectors before the model reads it.

Retrieval / RAG looking up a few passages in an external collection (index) and giving them to the model.

Top-k keeping only the k highest-scoring items (passages or cache entries).

ANN search approximate nearest-neighbor search: a fast lookup that can miss the true best match.

Agent memory notes, summaries or extracted facts an AI agent keeps about past steps or conversations.

Budget how much memory is allowed: tokens, cache entries, bits per entry, or retrieved items k.

Full cache the uncompressed reference run: the model reads everything.

Baseline / ablation a comparison run; an ablation removes one component to see what it contributed.

Exact match, F1, accuracy answer-quality scores; higher is better.

Perplexity how surprised the model is by text; lower is better.

Needle in a haystack a test that hides one fact in a long text and asks for it.

Abstract only we could read only the paper's summary, not the full paper (marked in the notes).

2. The rules (Sections 2.1 and 2.2 of the paper)

1

Behrouz et al.'s Miras framework treats Transformers, linear RNNs and Titans as associative memories; we extend this reading to inference-time and system-level stores. *Write* produces a store $\mathcal{S} = \{(\tilde{k}_i, \tilde{v}_i)\}_{i=1}^{n}$, *Store* holds it under a budget B, *Address* scores it against a query q, and *Read* combines the result. For softmax attention with key dimension d_k,

$$\alpha_i = \frac{\exp(q^\top \tilde{k}_i/\sqrt{d_k})}{\sum_j \exp(q^\top \tilde{k}_j/\sqrt{d_k})}, \qquad r(q, \mathcal{S}) = \sum_i \alpha_i \tilde{v}_i. \tag{1}$$

Each study is classified on four axes. The *operation* (W/S/A/R) is the step whose decision destroys or hides information; a study may carry several. The *storage substrate* is the representation written and stored. Native KV entries are the model's own context or KV cache; latent/soft vectors are learned continuous memories (e.g., soft prompts, gist tokens, trained KV prefixes). Natural-language text is content rewritten as text (e.g., compressed prompts, summaries, extracted facts, agent memory entries), however later retrieved. An external index is a retrieval corpus whose passages enter the prompt. Two further labels cover bounds on the sequence model (architecture) and cross-substrate frameworks (several). The *budget currency* is tokens, entries, bits per entry, rank or state dimension, or retrieved items k (distinct from keys k_i). For *query knowledge* at write time, a writer is *query-aware* (QA) if it conditions on the actual query, *proxy-query* (PQ) if it uses a stand-in such as an end-of-prompt observation window or past queries' attention, and *query-agnostic* (QG) if it has no query signal. *Multi-query reuse* (+R), where later queries read one store unchanged, is a read-side flag on one of these classes, never a standalone label; a reusable store is judged against all queries it serves. the paper's Table 1 gives each classified study's axes and term grades; "–" in its query-knowledge column marks a study without its own writer. Works cited only for context are not classified.

1

Capitalized names denote the terms below. *Capacity* is the loss of the best store that fits the budget, for a fixed reader and a writer with the stated query knowledge but no limit on computation; it is stated only as a bound or, absent one, as "capacity floor unknown", never as the best published method. At equal budget, a query-agnostic writer's capacity floor is no lower than a query-aware one's, a Capacity, not Addressing, effect: its effective per-query budget is roughly $B - H(Q)$ bits, with $H(Q)$ the query entropy.

Addressing: the needed content is stored, or would have fit within the budget, but the read does not reach it. It covers retrieval misses, positional under-use, stale or drifted keys, routing errors and selection shortfalls (dropping an item a better policy could have kept within the same budget).

Interference: stored content degrades the read of the answer-bearing item i^*, or perturbs reads with no answer-bearing item (off-context perturbation). Its sources are distractors, near-duplicate keys, crosstalk, learned compressed memories that derail off-context behavior, and superseded values read instead of or alongside the current value. *Key collision* is the near-duplicate case: distinct items receive keys the query cannot separate (the paper).

Write distortion: what is stored differs from what should have been, through quantization error, summarization drift, extraction loss (e.g., triples that lose context) or update failure (a required update omitted or written incorrectly).

Residual: loss remaining when i^* is present, intact and reached, and no change to competitors removes it, e.g., multi-hop reasoning failures with gold evidence in context, or degenerate generation. It is reported as residual, not forced into a term. As counterfactual attributions, terms need not sum to the total loss.

Addressing or Interference? Both appear as low read weight on i^*; intervention separates them. Addressing is the loss removed by raising access to i^* itself (keeping it at the same budget, or raising its read weight) beyond a

count-matched sham raising access to non-answer content. Interference is the loss removed by making competitors less similar to i^* (deleting, replacing with dissimilar items, or re-keying them) at the same competitor count, with i^* fixed. Deleting all competitors is not a valid test: a normalized read (softmax or top-k) then reaches i^* whatever its score. A change that does both (e.g., keeping i^* by dropping competitors) is Unassigned (Addressing, Interference) unless a control separates them.

Unlike the operation, the term records what a better store, policy, query path or writer would have recovered. An eviction policy decides at Store (Write for a compressor), but its selection shortfall is Addressing, written S→Addressing (operation→term). Key collision is Interference when near-duplicate keys or retained superseded values degrade the read, and Addressing when a duplicate displaces a unique item that would have fit.

The failure modes are: *Capacity saturation*; for Addressing, *retrieval miss*, *positional under-use*, *selection shortfall*, and *drift and staleness* of keys; for Interference, *key collision and redundancy*, *read-time crosstalk and dilution*, and *off-context perturbation*; and for Write distortion, the four causes above.

We apply these rules in order:

1. **Missing item.** If no store within the budget could hold the item for this writer's query-knowledge class, the loss is Capacity; if it would have fit but the policy dropped it, Addressing (selection shortfall). A partly evicted item counts as missing.
2. **Missing update.** A missing update mechanism or failed update is Write distortion (omission). If the retention policy evaluated and dropped the update within budget, rule 1 applies.
3. **Stale value.** If a superseded value is retained and read, the loss is Interference; if the current value was never written, rule 2 applies.
4. **Distorted, unreached or outcompeted item.** If the stored item is altered, the loss is Write distortion. If it is intact but not reached, the intervention above decides: Addressing if raising access to it repairs the read beyond a sham, Interference if less similar competitors at the same count do (e.g., key collision that pushes it out of the top-k).
5. **Policy comparison.** If one selection or eviction policy beats another at the same budget, query-knowledge class and task, the worse policy's Addressing shortfall is at least the gap. It is graded M only when the study shows, item by item, that keeping the answer-bearing item the worse policy dropped repairs its failures, and a count-matched control rules out gains from dropping competitors. Otherwise it is graded I, because the better policy may have won by reducing Interference.
6. **Cannot tell.** If a study does not separate the cases, the loss is Unassigned, with the confounded terms named.

Each cell carries one evidence grade. **M** (measured): the study's own intervention or ablation compares conditions that differ only in the term's factor, e.g., access to i^*, competitors' similarity at a fixed count, or how items are written at equal budget. **B** (bounded): a formal bound isolates the term. **I** (inferred): the evidence points to the term without isolating it; a method's or policy's overall gain, or altered items observed without a measured loss, is I. **U** (unassigned): the study's own comparison changes the term together with another, named, term. A "–" marks a term the study does not address; for Capacity, no loss is reported at a reduced budget, since a loss against a larger budget or the full store is U unless a bound applies, and changing bits per entry or rank also changes the budget. Capacity cannot be measured directly, so its cells are B, U or "–"; a store built within the budget gives only an empirical upper bound on the floor, which we report in the text but do not grade B. Studies read only in abstract are capped at I (U for Capacity).

Worked example. In our case study (the paper), SnapKV keeps 25% of the cache with the question in the context, so it is a query-aware writer. Take a query that the full cache answers correctly but the compressed cache does not: a C→W row (correct under the full cache, wrong as run). Rule 1 asks whether the answer-bearing record i^* is missing; part of it was evicted, so it counts as missing. Would it have fit? Rerunning the query with the tokens of i^* kept, at the same budget, gives the right answer, so a store within the budget that this writer could build holds it: the loss is Addressing (selection shortfall), not Capacity. Keeping i^* also displaces other entries, so the gain could come from fewer competitors; a count-matched sham, keeping a random non-answer span of the same length, repairs almost none of these rows, which rules that out. The repair is shown item by item against the

sham, so the cell is graded M. Without the question in the context, the protected writer would know more than its class allows, and the same repair would stay Unassigned (Capacity, Addressing).

3. Part A: studies to grade

1

A1. TCR-QF: How to Mitigate Information Loss in Knowledge Graphs for GraphRAG: Leveraging Triple Context Restoration and Query-Driven Feedback (2025) *Read in full text.* https://doi.org/10.24963/ijcai.2025/901

Summary. TCR-QF names two ways a GraphRAG knowledge graph loses information when text is turned into triples. "Context loss" means the triple drops semantic nuance and relational dependencies. "Information sparsity" means extraction misses triples a query needs. The first fix (TCR) attaches the most similar source sentence back onto each retrieved triple. The second fix (QF) has an LLM list the missing knowledge as sub-questions, retrieves passages with a dense retriever, extracts new triples, and adds them to the KG. This loop repeats up to 20 times. On five multi-hop QA benchmarks (gpt-4o-mini), it beats ToG, naive RAG and GraphRAG.

What they did. KG built by LLM triple extraction over 512-token chunks with 64-token overlap; each node keeps its source documents. Subgraph retrieval uses ToG-style LLM beam search. TCR: for each triple, the template "eh r et" is embedded and compared by cosine similarity with sentences from the head and tail entities' source documents, and the top sentence is appended (eh,r,et,S_top). QF: an LLM compares the query, the answer and the context to produce missing-knowledge sub-questions. Each sub-question drives dense retrieval (text-embedding-small) over the raw documents, the LLM extracts new triples, and the KG is updated after edit-distance deduplication. The loop stops when no knowledge is missing or after Imax = 20 iterations.

How they measured. Downstream QA Exact Match and F1 on 2WikiMultiHopQA, HotpotQA, MuSiQue-Full, MuSiQue-Ans (1,200 random samples each) and ConcurrentQA (1,600 samples). Also reported: an ablation (TCR alone, QF alone, answer-driven feedback) and KG node/edge counts per round. The amount of information lost is never measured directly; it is inferred from QA gains.

Findings.

- Over its GraphRAG competitors, TCR-QF averages a 29.1% EM gain and a 15.5% F1 gain. Evidence: "achieves a 29.1% improvement in Exact Match and a 15.5% improvement in F1 over its state-of-the-art GraphRAG competitors" (abstract)
- Restoring the source sentence for each triple, with no feedback loop, recovers a large share of the accuracy lost relative to bare triples (ToG). Evidence: "2Wiki EM 0.400 -> 0.481 (F1 0.476 -> 0.561); HotpotQA EM 0.420 -> 0.494 (F1 0.555 -> 0.642)" (Table 2 / Sec 4.2)
- Filling sparsity with query-driven feedback alone helps more than context restoration alone. Evidence: "QF(w/o TCR) 2Wiki EM 0.568, F1 0.651; HotpotQA EM 0.515, F1 0.656" (Table 2)
- Using the query to drive feedback beats extracting triples from the model's own answer, which the authors think spreads errors. Evidence: "TCR-AF 2Wiki EM 0.538 vs TCR-QF 0.598; "likely because it prevents error propagation from incomplete initial reasoning"" (Table 2 / Sec 4.2)
- The KG grows over the rounds and performance plateaus after roughly 8 rounds. Evidence: "Nodes 74,571 -> 78,661 (+4,090), edges 69,866 -> 80,472 (+10,606); EM 0.481 -> 0.598, F1 0.562 -> 0.680; stable from round 8" (Table 3 / Sec 4.3)
- Most of the recovered knowledge arrives in the first feedback round. Evidence: "Round 0 -> 1: edges 69,866 -> 74,006, EM 0.481 -> 0.555" (Table 3)
- TCR-QF beats hybrid GraphRAG on every dataset, with the largest relative gains on MuSiQue. Evidence: "MuSiQue-Full EM 0.303 vs 0.189; MuSiQue-Ans EM 0.366 vs 0.258; ConcurrentQA EM 0.492 vs 0.459" (Table 1)

Limitations.

- - **Proxy measurement only:** information loss is never quantified directly (e.g., the fraction of facts missing from the KG); it is only inferred from EM/F1. - **Narrow test setup:** results use a single backbone (gpt-4o-mini) and subsampled test sets. - **Limited ablations:** the ablation and convergence studies cover only 2Wiki and HotpotQA. - **Unreported costs:** latency and token cost of up to 20 iterations of LLM calls are not reported. - **One-sentence restoration:** TCR attaches only the single most similar sentence,

so context spread across several sentences may still be lost. - **Text/table mismatches:** - QF HotpotQA EM is 0.522 in the text but 0.515 in Table 2. - TCR-AF HotpotQA EM is 0.523 in the text but 0.531 in Table 2. - **Persistence unclear:** the KG grows within the test-time corpus, but the paper does not say whether the enrichment is kept across queries.

Your grades: Capacity _____ Addressing _____ Interference _____ Write distortion _____

1

A2. DBTrimKV: Make Each Token Count: Towards Improving Long-Context Performance with KV Cache Eviction (2026) *Read in full text.* `https://arxiv.org/abs/2605.09649`

Summary. DBTrimKV argues that keeping the full KV cache is not always best. In long contexts, irrelevant tokens soak up softmax attention mass that should go to the useful tokens ("attention dilution"). The method trains lightweight retention gates that predict a geometric-decay utility score for each KV entry. The gates' final projection is shared across all layers and heads, so scores are on one scale, and entries are ranked against each other under a single global budget covering every layer, head and modality. On VLM reasoning, multi-turn dialogue (MMDU) and LongBench-v2, it often matches or beats full-cache inference at small budgets. Heuristic baselines (SnapKV, R-KV, AdaKV) degrade sharply at the same budgets.

What they did. A per-layer, per-head retention gate g(x)=(w_gˆT Proj_{l,h}(x)+b_g) runs on the token embedding, with (w_g, b_g) shared across all gates. Its score is computed once, when the token enters the cache. The training loss is KL distillation from the full-cache teacher plus a global capacity penalty, with the base LLM kept frozen. At inference, entries are ranked by the lookahead score G=ˆ{t+1-i}(1-ˆ{T-t})/(1-), with a default T-t=2. The top M_global entries across all layers and heads are kept. Variable-length caches for each head are handled with PagedAttention.

How they measured. Benchmarks: accuracy on GQA, VQAText, MME, MMStar, MathVision, MMMU-Pro, VideoMME, VideoMMMU, VideoMathQA, AIME24, GSM8K, MATH-500 and LongBench-v2, plus MMDU scored by an LLM judge (Gemini). Results are reported as % of vanilla full-cache performance. The authors also run a multi-key/multi-value/multi-query needle-in-a-haystack test and a token-survival analysis (top-K and mass criteria against horizon L). Efficiency is reported as decoding time and throughput.

Findings.

- Under full-cache inference, a representative head attends to few old tokens at long horizons. Evidence: "only 13.8% of tokens survive to L = 4096 under top-K = 256, and only 3.2% survive to L = 16k" (Section 3.2, Figure 2)
- Many near-tie distractors force the attention mass on useful tokens toward zero. Evidence: "if = O(1), |Ut| = O(1), and |Dt| →, then t →1" (Proposition 3.1)
- DBTrimKV holds or beats full-cache performance on VLM reasoning at budgets where baselines collapse. Evidence: "At budget 128: DBTrimKV 98.27% vs vanilla; SnapKV 35.58, AdaKV 37.30, TrimKV 68.13; at 512 DBTrimKV 103.73" (Table 2)
- On multi-turn dialogue, DBTrimKV beats full-cache inference at every budget tested. Evidence: "114.46% at 512, 108.33% at 256, 104.10% at 128; SnapKV 64.00 at 128" (Table 3 / Table 6)
- Heuristic eviction loses visually grounded context in dialogue. Evidence: "Visual Perception at 128: SnapKV 1.72, DBTrimKV 3.54 vs Vanilla 3.40" (Table 6, Appendix C.1)
- Gains carry over to long-context text QA. Evidence: "DBTrimKV avg acc 31.44 vs Full KV 28.79 (+9.20%); TrimKV -9.24%, Locret -1.32% at budget 1024" (Table 7, Appendix C.2.1)
- Sharing the final projection across gates is necessary for global eviction to work. Evidence: "MathVision budget 256: w/o tying 40.13 vs with tying 51.64 (vanilla 48.68)" (Table 4)
- Using the token embedding as gate input beats using concatenated KV. Evidence: "256 budget: [k||v] 43.75 vs x_t 51.64" (Table 5)

- The best lookahead horizon depends on the budget. Evidence: "At 1024: Tt=2 52.63 vs Tt=5 46.05; at 64: Tt=5 38.49 vs 33.88" (Table 8, Appendix C.3)
- The model keeps vision boundary tokens as structural anchors while evicting fine-grained patches. Evidence: "the model persistently retains the boundary tokens <|vision_start|> ... and <|vision_end|> ... across numerous heads" (Section 5.2, Figure 6)

Limitations.

- Only the gates are trained; the base model is frozen. The gates need training data for each base model, while heuristic baselines need no training. Gains over vanilla are smaller on short-reasoning tasks. Results are reported for a fixed default lookahead, which is not best at every budget. Dynamic budgets add a small runtime overhead compared with TrimKV. Efficiency was measured on synthetic filler inputs. MMDU is scored by an LLM judge. The provided text is cut off partway through the appendix's limitations section (Appendix D).

Your grades: Capacity _____ Addressing _____ Interference _____ Write distortion _____

1

A3. ICAE: In-context Autoencoder for Context Compression in a Large Language Model (2024) *Read in full text.* `https://arxiv.org/abs/2307.06945`

Summary. ICAE compresses a context into k learned memory slots. The encoder is a LoRA-adapted LLM that adds learnable memory tokens. The decoder is the frozen target LLM, which conditions on the slots in place of the original text. The model is pretrained with autoencoding plus language modeling on the Pile, then fine-tuned on the PWC instruction dataset. It reaches about 4× compression (512→128) with near-lossless reconstruction. Loss grows as the compression ratio rises, as context length goes past the slot capacity, and when the text is unpredictable (random). Larger or stronger base LLMs compress better.

What they did. k memory tokens are appended to the context (L tokens) and passed through a LoRA-adapted encoder (LoRA on the Q/V projections). Their outputs become the memory slots, which the frozen LLM decodes. Pretraining loss is ·L_AE + (1)·L_LM, followed by instruction fine-tuning on (context, prompt, response) data. For long inputs, chunks are compressed separately and the resulting spans are concatenated. Adds about 1% parameters.

How they measured. Autoencoding: BLEU-4, exact-match prefix ratio, and cross-entropy loss, broken down by context length. Continuation perplexity (PPL) with memory slots vs. with the original context. GPT-4 pairwise win/lose/tie judgments on the PWC test set (240k train / 18k test). Latency and GPU-memory profiling.

Findings.

- At 4× compression with k=128, reconstruction is near-lossless for contexts up to about 300 tokens and degrades beyond 400. Evidence: "When the context length is within 300, the ICAE can almost perfectly reconstruct the original context... even at a context length of 500, the median BLEU remains over 0.98, and the median EM approaches 0.6" (Section 3.2.1, Figure 4)
- Going above 4× compression is hard. Evidence: "the BLEU scores become much less satisfactory for k values of 64 and 32, indicating an inability to losslessly retain the original context... achieving over 4× compression is rather challenging" (Section 3.2.1, Figure 5)
- The perplexity penalty grows with the compression ratio. Evidence: "PPL delta +0.16 (1×), +0.32 (2×), +0.49 (4×)" (Table 1)
- How much is retained depends on how predictable the content is: random text is almost entirely lost. Evidence: "Normal text BLEU 99.3; Patterned random 3.5; Completely random 0.2 (loss 0.01 / 1.63 / 4.55)" (Table 3)
- Stronger base LLMs lose less under the same 512→128 compression. Evidence: "PPL +0.49 (Llama-7b), +0.37 (Llama-2-7b), +0.30 (Llama-2-13b); AE BLEU 99.1/99.5/99.8" (Table 6)

- On downstream QA, memory slots still fall short of the original context, and the gap narrows as k grows. Evidence: "Llama-2-7b-chat ICAE vs original context: win+tie 48.4 (k=64), 54.6 (k=128), 77.8 (k=256)" (Table 4)
- Pretraining matters: with pretraining, 8× compression matches 4× compression without it. Evidence: "k = 64 (pretrained) VS k = 128 (no pretraining): 33.1 win / 28.0 lose; k=128 pretrained vs no pretraining win/lose 6.4" (Table 5)
- At equal length, memory slots carry more information than a natural-language summary. Evidence: "k = 128 (pretrained) VS 128-token summary (by GPT-4): 34.1 win, 17.6 lose, 48.3 tie, win/lose 1.9" (Table 5)
- Compression speeds up inference and saves GPU memory. Evidence: "8*2048: 24.0 → 7.3 (3.3×); 32*512: 24.3 → 6.8 (3.6×); 2048 memory slots instead of 4096 tokens can save about 20GB of GPU memory" (Table 7; Section 3.3.3)
- Concatenating separately compressed spans only works after training on concatenated examples. Evidence: "this did not work initially, because the model had never seen multiple span concatenation patterns during training" (Section 3.3.3)

Limitations.

- Experiments only use Llama models up to 13B. Most results use a maximum of 512 tokens. Downstream quality is judged by GPT-4 on the authors' own GPT-4-built PWC dataset. The memorization analogy to humans rests on a single qualitative example. Results for multi-span long contexts are reported only as perplexity (Figure 6). There is no comparison against standard long-context benchmarks or other compressors such as Gist or AutoCompressor.

Your grades: Capacity _____ Addressing _____ Interference _____ Write distortion _____

1

A4. RULER: RULER: What's the Real Context Size of Your Long-Context Language Models? (2024) *Read in full text.* `https://arxiv.org/abs/2404.06654`

Summary. RULER is a configurable synthetic benchmark (13 tasks across retrieval/NIAH variants, multi-hop variable tracking, aggregation, and QA) showing that 17 long-context LMs, despite near-perfect vanilla NIAH scores, degrade sharply with length. Only about half reach an 'effective length' of 32K, and almost all fall below threshold before their claimed context size.

What they did. Synthetic tasks with controllable sequence length and complexity: number and type of needles, distractor keys, multiple values or queries, hop and chain counts, word-frequency distribution, and distractor paragraphs in QA. Effective length is the longest context whose average score exceeds Llama2-7B's 4K score (85.6%).

How they measured. Recall-based accuracy on 500 examples per task per length (4K–128K). Metrics: effective vs. claimed length, and weighted averages across lengths (weights increasing or decreasing with length). Yi-34B error analysis extends to 256K.

Findings.

- Near-perfect NIAH scores hide large degradation on RULER. Evidence: "Despite achieving nearly perfect accuracy in the vanilla NIAH test, almost all models exhibit large performance drops as the context length increases." (abstract)
- Only half of the models are effective at 32K, although all claim at least 32K. Evidence: "While these models all claim context sizes of 32K tokens or greater, only half of them can maintain satisfactory performance at the length of 32K." (abstract)
- GPT-4 degrades from 4K to 128K, and its effective length is half its claimed length. Evidence: "GPT-4 claimed 128K, effective 64K; 96.6 at 4K vs 81.2 at 128K" (Table 3)
- Models claiming 1M context can be effective at only 16K or less. Evidence: "LWM (7B) 1M claimed, effective <4K; GradientAI/Llama3 (70B) 1M claimed, effective 16K" (Table 3)

- Yi-34B scores perfectly on vanilla NIAH but only 77.3 on the full RULER average at 128K. Evidence: "Yi (34B) vanilla NIAH 100.0 at 128K; RULER 77.3 at 128K" (Table 11 and Table 3)
- Hard distractors cause large losses at long context. Evidence: "Yi dropping by 40 points at 256K in the extreme version, where the context is full of irrelevant needles" (Section 5)
- Multi-item retrieval returns incomplete information. Evidence: "increasing the number of queries from 1 to 8 drops the performance by 15 points" (Section 5)
- At long lengths, Yi copies from the in-context demonstration instead of solving the task. Evidence: "Over 80% of Yi's output in the CWE task at 128K is simply a string copied from the one-shot example" (Section 5)
- In long-context QA, Yi's performance approaches its no-context baseline. Evidence: "Yi's performance approaches its no-context baseline as we extend the context with distracting paragraphs" (Section 5)
- Aggregation degrades much faster than retrieval as length grows. Evidence: "Llama3.1 (8B) aggregation 97.0 at 4K vs 36.2 at 128K" (Table 15)
- Performance drops abruptly beyond the training length. Evidence: "abrupt performance drops when models need to extrapolate to unseen lengths ... and almost linear degradation with input length on log scale within the max training context size" (Section 6)
- Non-Transformer models lag behind the Transformer baseline. Evidence: "both underperform the Transformer baseline Llama2-7B by large margins up till the length of 4K" (Section 6)

Limitations.

- No position/depth control, so lost-in-the-middle effects are not measured. The synthetic proxies are not validated against realistic tasks. Tasks were chosen to be easy at 4K. Prompt robustness and some hyperparameters were not studied.

Your grades: Capacity _____ Addressing _____ Interference _____ Write distortion _____

1

A5. LLMLingua: LLMLingua: Compressing Prompts for Accelerated Inference of Large Language Models (2023)
Read in full text. `https://arxiv.org/abs/2310.05736`

Summary. LLMLingua compresses prompts for black-box LLMs using a small LM's perplexity. It has three parts: a budget controller that allocates compression ratios across instruction, demonstrations, and question and drops whole demonstrations; iterative token-level compression that conditions on already-compressed segments; and instruction-tuning alignment of the small LM to the target LLM. It reaches up to 20x compression on GSM8K with little EM loss.

What they did. Coarse-to-fine perplexity-based token and demonstration dropping with a small LM (Alpaca-7B or GPT2-Alpaca). Demonstrations are ranked by perplexity and kept within budget k·_dems·L_dems. Segments of 100 tokens are then compressed iteratively, with each segment conditioned on the previously compressed segments. Tokens with PPL above a dynamic threshold _j are kept. The small LM is instruction-tuned on LLM-generated (Alpaca) data for distribution alignment.

How they measured. Exact Match on GSM8K and BBH. BLEU, ROUGE, and BERTScore F1 on ShareGPT and Arxiv-March23. Performance is measured under compression-ratio constraints (1-shot, half-shot, quarter-shot, token constraints) against Selective-Context, Sentence Selection, and GPT-4 generation. Also reported: ablations, end-to-end latency on a V100, and GPT-4 reconstruction of compressed prompts.

Findings.

- At 20x compression on GSM8K, EM stays near full-shot; Selective-Context degrades heavily at a similar ratio. Evidence: "Full-shot 78.85 (2,366 tokens); Ours quarter-shot 77.33 (117 tokens, 20x); Selective-Context 44.20 (15x)" (Table 2)

- At 5x compression on GSM8K (1-shot constraint), LLMLingua slightly exceeds full-shot EM. Evidence: "Ours 79.08, 446 tokens, 5x vs Full-shot 78.85" (Table 2)
- BBH is more sensitive to compression than GSM8K. Evidence: "On BBH, our approach achieves compression ratios of 5x and 7x with the EM score decreasing by 8.5 and 13.2 points" (Section 5.2)
- Iterative, conditioned token compression matters: single-pass compression loses information. Evidence: "w/o Iterative Token-level Prompt Compression 72.93 vs Ours 79.08" (Table 3)
- The budget controller, and perplexity-based rather than random demonstration selection, preserve information. Evidence: "w/o Budget Controller 73.62; w/ Random Selection in Budget Controller 72.78" (Table 3)
- Distribution alignment gives only a small gain. Evidence: "w/o Distribution Alignment 78.62" (Table 3)
- GPT-4 abstractive compression omits reasoning details. Evidence: "GPT4 Generation 56.33 at 20x on GSM8K; 26.81 on BBH" (Table 2)
- All methods collapse at extreme compression ratios. Evidence: "notable performance drop when trying to achieve excessively high compression ratios such as 25x-30x on GSM8K" (Limitations / Figure 3)
- Recoverability depends on the small LM: GPT-4 restores the full reasoning from Alpaca-7B compression but only a partial, erroneous version from GPT2 compression. Evidence: "restored to its complete 9-step reasoning process, while ... GPT2-Alpaca can only be restored to a 7-step reasoning process, with some calculation errors" (Section 5.5)
- LLMLingua delivers an end-to-end speedup with small compression overhead. Evidence: "speedup ranging from 1.7x to 5.7x" (Table 6)

Limitations.

- Performance drops sharply at 25x-30x compression. The upper limit on compression varies with prompt length, task type, and number of sentences. Tokenizer mismatch between the small LM and the target LLM may underestimate prompt token length. Distribution discrepancy persists even with alignment (GPT2-Alpaca is 2.06 EM worse at 1-shot). Claude experiments were limited by API cost.

Your grades: Capacity _____ Addressing _____ Interference _____ Write distortion _____

1

A6. Łajewska et al.: Understanding and Improving Information Preservation in Prompt Compression for LLMs (2025) *Read in full text.* `https://doi.org/10.18653/v1/2025.findings-emnlp.949`

Summary. The paper proposes an evaluation framework for prompt compression that looks past downstream accuracy to two more things: whether responses stay grounded in the source (scored with FABLES) and how much information survives (measured by reconstructing the context and scoring the reconstruction with BERTScore, ROUGE and entity match). It applies the framework to LLMLingua (hard compression) and to xRAG and PISCO (soft compression), all with Mistral-7B as the target model. Each method loses information. xRAG's single token mostly keeps the topic and drops details such as dates and numbers. Changing the granularity of xRAG's training (sentence-level and two-step pre-training) gives up to +23% downstream, up to +8 BERTScore points in grounding and 2.7× more entities preserved. PISCO, trained with sequence-level distillation, gets the best trade-off between compression and effectiveness.

What they did. Soft compression: xRAG maps a frozen dense-retrieval embedding into the LLM's embedding space as one token, through a trainable bridge. PISCO uses encoder and decoder adapters with 8 embeddings, trained by sequence-level knowledge distillation. Hard compression: LLMLingua prunes tokens to a 350-token budget. The proposed fix trains xRAG at finer granularity: single-sentence pre-training, then multi-token pre-training, then fine-tuning with one token per sentence.

How they measured. Downstream: exact match (HotpotQA, TriviaQA, GSM8K) and BERTScore F1 (arXiv summarization, QuAC). Grounding: FABLES claim-level faithfulness, judged by Claude 3 Haiku. Information preservation: the target LLM reconstructs the context from the soft prompt, scored with BERTScore and ROUGE

against the original, plus the fraction of spaCy-detected entities that survive. Also reports compression rate in tokens and compute cost in FLOPs.

Findings.

- All compression methods lose the most on multi-hop HotpotQA. Evidence: "HotpotQA EM: Mistral-7B 0.664; xRAG 0.297 (-55%); PISCO 0.318 (-52%); LLMLingua 0.306 (-54%)" (Table 2)
- Compressed in-context examples do worse than giving no examples at all. Evidence: "GSM8K EM: no cont. 0.440; xRAG 0.336; LLMLingua 0.305" (Table 2, Section 4.1)
- Compression lowers grounding a lot, most of all for xRAG. Evidence: "FABLES arXiv-sum.: Mistral-7B 0.97, xRAG 0.39, PISCO 0.74, LLMLingua 0.62; QuAC 0.93 vs xRAG 0.45" (Table 4)
- xRAG cannot fully reconstruct the context, and splitting into one token per sentence makes it worse. Evidence: "Unseen data BERTScore F1 (All): 1 per sample 0.66 vs 1 per sent. 0.42; preserved entities 0.28 vs 0.19" (Table 5)
- PISCO keeps much more information than xRAG. Evidence: "PISCO unseen BERTScore 0.89 (8 per sample) / 0.90 (8 per sent.); entities 0.49 / 0.59" (Table 5)
- Dates and numbers are lost most often; locations are kept best. Evidence: "xRAG unseen 1 per sentence: DATE 0.14, CARD. 0.13, GPE 0.25" (Table 7, Section 4.3)
- Reconstruction introduces hallucinated facts. Evidence: ""In May 1983, she married Nikos Karvelas..." is reconstructed as "In May 1985, she married Nikos Karvelas, a composer and lyricist."" (Appendix E.3)
- Two-step pre-training raises entity preservation and similarity when the context is encoded at sentence level. Evidence: ""we can reconstruct 50% of entities ... (compared to 13% for the xRAG (Reproduced))"; "F1 of 0.63 vs. 0.19"" (Section 5.2, Figure 2)
- Training at finer granularity improves downstream results on HotpotQA*. Evidence: ""23% relative improvement over the baseline (in EM, from 0.38 for xRAG (Reproduced) Cont. to 0.47 for xRAG w/ Two-Step PT + FT Sent.)"" (Section 5.2, Table 6)
- PISCO compresses about as much as single-token xRAG but performs better. Evidence: "HotpotQA tokens: Mistral 1515; xRAG 65 (23.3); PISCO 73 (20.8); LLMLingua 388 (3.9)" (Table 10)
- Granularity matters differently for each method: finer units hurt xRAG and help PISCO on HotpotQA. Evidence: "HotpotQA EM xRAG 1 per sentence 0.055 (-81%); PISCO 8 per paragraph 0.512 (+61%)" (Table 3)

Limitations.

- Only one target LLM (Mistral-7B) and English-only benchmarks. One hard method and two soft methods. Grounding relies on an LLM judge (Claude 3 Haiku). Reconstruction-based measurement depends on the prompt; the authors say different prompts give widely varying content. The xRAG reproduction used about 50% of the original fine-tuning data. Information preservation was not measured for LLMLingua.

Your grades: Capacity _____ Addressing _____ Interference _____ Write distortion _____

1

A7. CCM: Continuous Context Management (2026) *Read in full text.* `https://arxiv.org/abs/2609.35540`

Summary. Studies Continuous Context Management (CCM): at every turn the agent emits an updated memory plus action, and the next prompt contains only task, memory, and newest observation. Out-of-box on TerminalBench-2 with four frontier models, CCM cuts input tokens ~76-84% but lowers success for 3 of 4 models. Proposes GRPO plus gated privileged full-history self-distillation (frozen initial model scores student actions under reconstructed full history) to train open-weight CCM agents on WebShop and Endless Terminals.

What they did. Per-turn model-written memory replaces the transcript: (m_n, a_n) ~ pi(x, m_{n-1}, o_{n-1}). Training uses GRPO on memory and action tokens, plus an SDAR-style distillation loss on action tokens

only. The loss is gated by sigmoid(beta*Delta), where Delta is the gap between the frozen full-history teacher's and the compact-context student's log-probs (lambda=0.01, beta=5).

How they measured. Measurement covers several dimensions: pass@1 success, and peak prompt tokens (median/max). It also covers cumulative input tokens, API cost including prompt caching, and memory update rate. Further measures are prefix retention, memory size, episode turns, and general-capability retention (MMLU-Pro, HellaSwag, IFEval).

Findings.

- CCM sharply reduces cumulative input tokens without training. Evidence: "reduces mean cumulative input tokens per task by 83.1% for Claude Sonnet 4.6, 81.5% for Claude Opus 4.6, 84.3% for GLM-5, and 75.8% for Kimi K3" (Section 3.2)
- Compression costs task success for most frontier models; Kimi K3 is the exception. Evidence: "decreases of 11.2, 18.0, and 5.6 percentage points, respectively. In contrast, Kimi K3 improves slightly under CCM, from 62.9% to 64.0%" (Section 3.2 / Table 1)
- Retained memories are small, but peak prompts remain large because observations are uncompressed. Evidence: "The largest retained CCM memories contain only 561–1,403 tokens... newest terminal observation, which may contain up to 60,000 characters" (Section 3.2)
- Fewer tokens do not imply lower cost under prefix caching. Evidence: "$0.831 versus $1.087 for Opus and $0.302 versus $0.622 for Kimi" (Section 3.2 / Table 1)
- Adding privileged distillation greatly improves trained CCM on WebShop; for the 4B model it beats full-history GRPO. Evidence: "improves exact success by 62.50 percentage points, from 24.22% to 86.72%... also exceeds full-history GRPO, which achieves 75.78%" (Section 4.2)
- For Qwen3-8B on WebShop, full history remains best. Evidence: "Full-history GRPO remains the strongest condition for this model, reaching 80.47% exact success" (Section 4.2)
- GRPO-only CCM policies rarely update memory; distillation yields active memory management. Evidence: "memory-update rates of 12.9% and 9.1%... Qwen3-4B-Instruct changes its memory on every consecutive turn while retaining 77.2% of the preceding prefix" (Section 4.2 / Table 2)
- On Endless Terminals, CCM loses little, possibly because state is externalized in the sandbox. Trained CCM still trails full-history GRPO. Evidence: "solving 20.33% and 21.00%... CCM + GRPO + distillation 32.00; Full-history GRPO 35.33" (Section 4.3 / Table 3)
- Distillation alone, without GRPO, fails: memory tokens get no signal. Evidence: "Qwen3-4B-Instruct solves no tasks, while Qwen3-8B solves only one of 128" (Appendix D.1 / Table 7)
- Distillation reduces forgetting of general capabilities for the 4B model. Evidence: "MMLU-Pro 64.23 (1.03) vs CCM + GRPO 55.16 (10.09)" (Table 4)

Limitations.

- Evaluation is limited to a few benchmarks and models. Trained results use only Qwen3-4B/8B. Gains are inconsistent across scales and environments. Observations are not compressed. Peak-prompt statistics are omitted on Endless Terminals. API-cost results depend on provider caching. The OPD connection is local and approximate. There is no comparison to threshold-based compaction baselines.

Your grades: Capacity _____ Addressing _____ Interference _____ Write distortion _____

1

A8. KVDiagnosis: KVDiagnosis: A Diagnostic Benchmark for KV-Cache Compression in Long-Context Language Models (2026) *Read in full text.* `https://arxiv.org/abs/2608.09412`

Summary. KVDiagnosis is a paired diagnostic benchmark for KV-cache compression. Each source runs once under FullCache. Every supported method–setting cell then runs on all sources. FullCache-correct/compressed-wrong (C→W) rows are extracted per cell only after the full matrix is complete. Each row is linked to cache-

retention, likelihood, attention and decoding diagnostics, with explicit N/A states. On Qwen3-8B the benchmark covers 8 methods spanning 5 mechanism families and four workloads (RULER-8K/16K, Qasper, HotpotQA).

What they did. Eight compressors are evaluated: StreamingLLM, SnapKV, TOVA, KeyDiff, AdaKV, ThinK, ChunkKV and QuantizedCache (HQQ), at 75/50/25% retention or 8b/4b/2b precision. Evidence spans are aligned after final-prompt tokenization. Diagnostics are measured on the failure rows: - Retention: ERR and ECov, averaged per layer–KV-head slot. - Predictive drift: NLL, GPR, KL, Top-50 overlap, gold-rank shift. - Access: EAM, EAR, NEAE. - Decoding traces. Failures are classified into six predeclared diagnostic categories. A causal test boosts attention logits on retained evidence positions by log 4.

How they measured. Task score, the C→W rate over FullCache-correct sources, the four correctness transitions, and failure-set Jaccard between methods. Stratified AUROC compares C→W rows against C→C controls. An intervention compares an evidence-attention boost against a count-matched sham, with a C→C safety check. The C→W/coverage analysis is repeated on Falcon3-7B and Mistral-24B.

Findings.

- Most failures show low or partial evidence coverage. Evidence: "63.2% have low or partial measured/projected coverage" (abstract)
- Drift with high mapped coverage is rare. Drift under structural addressability (ThinK/quantization) is common. Evidence: "Only 19 rows (0.2%) ... another 2,126 (17.0%) preserve structural position addressability" (abstract; Table 4)
- All ten diagnostics separate failed from successful compression. Evidence: "stratified AUROC 0.684–0.871" (abstract; Figure 6)
- A targeted evidence-attention boost selectively repairs low-EAR failures. Evidence: "repairs 28/96 (29.2%); ... non-evidence positions repairs 6/96 (6.3%) ... degrades only 3/92 (3.3%) matched C→C controls" (Section 5.3)
- Aggregate scores hide large differences in failure rates at light compression. Evidence: "TOVA, AdaKV, SnapKV, and QuantizedCache stay within 0.3 points, but their C→W rates span 0.8–2.0%" (Section 5.2)
- Methods with similar mean scores fail on different sources. Evidence: "mean scores of 84.8 and 85.7. Yet 11 of 12 workload–setting cells have failure-set Jaccard overlap at or below 0.385" (Section 5.4)
- Channel pruning and 2-bit quantization collapse, with severe likelihood drift. Evidence: "ThinK at 50% flips all 951 FullCache-correct sources with mean NLL 7.958 and GPR 0.015. QuantizedCache at 2b flips 883 of 951" (Section 5.3)
- Coverage loss grows as retention decreases. Evidence: "StreamingLLM falls from 0.262 to 0.074 and TOVA from 0.951 to 0.445 between 75% and 25%" (Section 5.3)
- Coverage diagnoses transfer across models, but quantization drift is model-specific. Evidence: "slot-ECov correlates with Qwen at = 0.969 for Falcon and 0.957 for Mistral ... QuantizedCache ... 2.153 on Qwen versus 0.348 and 0.021" (Section 5.5)

Limitations.

- - Primary model is Qwen3-8B; attention AUROC and interventions are Qwen-only. - Greedy decoding and fixed splits. - ERR/ECov are N/A for structural methods, so representation fidelity remains unmeasured. - EAR is valid for only 56.2% of rows. - AUROC intervals are descriptive because sources recur. - QA scorer is not the official one. - No byte equivalence across mechanisms. - Latency and memory are out of scope. - Categories are observational, not causal.

Your grades: Capacity _____ Addressing _____ Interference _____ Write distortion _____

1

A9. PI-LLM: Unable to Forget: Proactive Interference Reveals Working Memory Limits in LLMs Beyond Context Length (2025) *Read in full text.* `https://arxiv.org/abs/2506.08184`

Summary. PI-LLM benchmark adapts cognitive-science proactive interference: models read streams of repeated key–value updates and must report only the final value per key. Across 35 LLMs, accuracy falls log-linearly toward zero as the number of updates per key, number of keys, or value length grows. This holds even at fixed input length. Errors are mostly overwritten earlier values. Model size, not context window, predicts robustness. Natural-language 'forget' prompts barely help.

What they did. Synthetic key–value update stream (46 categories × 400 words). Interference load is varied along four axes: update count per key (3–400), number of updated keys, number of tracked keys at fixed length, and value length. The Interference Endurance Score (IES) is the AUC of accuracy over log-scaled update counts. Mitigations tested: per-key forget, forward focus, relevance meta-prompt, soft session reset, and mock QA reset.

How they measured. Fraction of tracked keys whose final value is retrieved correctly, with bootstrapped 95% CIs. Error position histograms bin responses by position in the update sequence and separate off-values (hallucinations) and off-keys. IES is regressed against size class and context length.

Findings.

- Accuracy declines log-linearly with interference across all models, reaching zero for mid-size models at high update counts. Evidence: "at 400 updates per key, the L models maintain relatively high accuracy, whereas the M group's accuracy drops sharply to 0%—with only one exception" (Section 2.2)
- Model size predicts interference robustness; context window length does not. Evidence: "parameter size class is a significant predictor of IES ($t = 3.03$, $p = 0.005$, $N = 30$), while context length has no significant effect ($t = -0.144$, $p = 0.886$)" (Section 2.2.2)
- Within models of similar context length, size strongly correlates with IES. Evidence: "$2 = 0.673$, $p = 0.0016$" (Section 2.2.2 / Figure 6)
- Interference degrades retrieval even at fixed input length when only the number of tracked keys varies. Evidence: "llama4-maverick achieves nearly 100% accuracy when tracking just two keys, but this drops below 5% when tracking 46 keys" (Section 3.2.2)
- Longer values sharply increase interference. Evidence: "increasing value length from one to ten words drives accuracy below 40% for every model tested, and extending it to forty words reduces accuracy to under 5%" (Section 4.1)
- Errors shift from adjacent prior values to distant early values and then to hallucinations as load grows. Evidence: "a substantial portion of errors remains anchored to the earliest bins, reflecting a persistent primacy bias toward the first few updates for each key" (Section 2.2.1)
- Per-key forget prompts fail and relocate errors to the injection point. Evidence: "$<$ 10 percentage points compared to the baseline at 100 updates" (Section 5.2 / Figure 12, 13)
- The mock QA reset hack substantially helps, but accuracy still declines as updates grow. Evidence: "Gemini Flash 2.0's retrieval accuracy at 150 update count—under high interference—matched its baseline performance at only 30–45 update count" (Section 5.3)
- Under strictly sequential (non-shuffled) updates, failure is step-like rather than gradual. Evidence: "retrieval accuracy remains stable until reaching a model-specific interference threshold, after which performance sharply and consistently drops to near-zero" (Section 2.2 / Figure 4)
- Mixture-of-Experts (MoE) models underperform dense models of comparable total size. Evidence: "MoE architectures consistently match or underperform dense models with similar total parameter counts" (Figure 7)
- Chain-of-thought (CoT) does not improve retrieval under interference. Evidence: "In three of the four comparisons, the CoT variant performs worse than or equal to its base model." (Figure 14)

Limitations.

- The task is purely synthetic. The regression on size vs. context length explains only 26.1% of variance and is restricted to open models with coarse size classes. Reasoning models were excluded from the regression, and CoT variants were only tested up to about 100 updates. The mock QA reset discards all prior context,

so it is not a general fix. Human comparison data are referenced but not reported in detail. The working-memory analogy is interpretive, and there is no mechanistic analysis of attention or the KV cache.

Your grades: Capacity _____ Addressing _____ Interference _____ Write distortion _____

1

A10. RGB: Benchmarking Large Language Models in Retrieval-Augmented Generation (2024) *Read in full text.* `https://arxiv.org/abs/2309.01431`

Summary. Introduces RGB, a bilingual (English/Chinese) RAG benchmark built from recent news with search-engine-retrieved documents, split into four testbeds (noise robustness, negative rejection, information integration, counterfactual robustness), and evaluates 6 LLMs. Models tolerate moderate noise but degrade sharply at high noise ratios, rarely reject when evidence is absent, struggle to integrate multi-document answers, and trust counterfactual retrieved content over their own knowledge.

What they did. Retrieval-augmented generation: 5 external documents per question (chunks of max 300 tokens from Google API pages, reranked by a dense retriever), with controlled ratios of noisy (relevant but answer-free) vs positive documents, or with manually injected factual errors.

How they measured. Exact-match accuracy (noise robustness, integration). Rejection rate by exact match and by ChatGPT judge (Rej/Rej*). Error detection rate (ED/ED*) and error correction rate (CR) for counterfactual docs. 600 base questions plus 200 integration and 200 counterfactual questions, half English and half Chinese.

Findings.

- Accuracy drops substantially as noise ratio rises to 0.8. Evidence: "the performance of ChatGPT has decreased from 96.33% to 76.00%, while the performance of ChatGLM2-6B has decreased from 91.33% to 57.33%" (Results on Noise Robustness / Table 1)
- Negative rejection is weak even under the lenient ChatGPT-judged metric. Evidence: "The highest rejection rates for LLMs in English and Chinese were only 45% and 43.33%, respectively." (Results on Negative Rejection / Table 3)
- Models do not reliably follow the instructed rejection string, so strict exact-match rejection is far lower than judged rejection. Evidence: "ChatGPT Chinese Rej 5.33 vs Rej 43.33" (Table 3)
- Multi-document integration is poor even without noise and worsens with noise. Evidence: "Even without noise, the highest accuracy of LLMs can only reach 60% and 67% for English and Chinese, respectively. After adding noise, the highest accuracy decreases to 43% and 55%." (Results on Information Integration / Table 5)
- Complex questions degrade at lower noise than simple ones. Evidence: "Performance decline becomes significant when the noise ratio is 0.4, but for simple problems, a significant decline occurs only at a noise ratio of 0.8" (Results on Information Integration)
- Integration errors in ChatGLM2-6B break down into merging, ignoring, and misalignment errors, alongside noise-type errors. Evidence: "Merging Error (28% of the total)... Ignoring Error (28% of the total)... Misalignment Error (6% of the total)" (Results on Information Integration, Error Analysis)
- Counterfactual documents override correct internal knowledge, and errors are rarely detected. Evidence: "ChatGPT-zh Acc 91, Accdoc 17, ED 1; ChatGPT-en Acc 89, Accdoc 9, ED 8" (Table 7)
- Noise errors stem from long-distance information, evidence uncertainty, and concept confusion. Evidence: "the model answer focuses on the concept "automotive revenue" in the document rather than "revenue" in the question" (Results on Noise Robustness, Error Analysis / Table 2)

Limitations.

- Only 6 models, mostly 6-7B plus gpt-3.5-turbo. Fixed 5 documents per question, so no long-context scaling is tested. Exact-match metrics and a ChatGPT judge are used. Counterfactual testing covers only 3 model-language settings above 70% no-doc accuracy, and only the setting with warnings in the instruction. Questions are ChatGPT-generated from news. The integration error analysis covers a single model.

Your grades: Capacity _____ Addressing _____ Interference _____ Write distortion _____

1

A11. Jiang et al.: Anatomy of Agentic Memory: Taxonomy and Empirical Analysis of Evaluation and System Limitations (2026) *Read in full text.* `https://arxiv.org/abs/2602.19320`

Summary. Survey plus empirical study of agentic memory (Memory-Augmented Generation). Proposes a four-way structural taxonomy: Lightweight Semantic, Entity-Centric/Personalized, Episodic/Reflective, and Structured/Hierarchical. It then benchmarks six systems (AMem, MemoryOS, Nemori, MAGMA, SimpleMem, MemSkill) on LoCoMo along four axes: benchmark saturation, lexical vs. LLM-judge metric validity, backbone sensitivity (format errors that corrupt memory writes), and latency/construction cost.

What they did. External, writable, non-parametric memory state M_t. The agent reads it via query-conditioned retrieval (top-k similarity, graph traversal, hierarchical search) and updates it through memory actions (STORE, UPDATE, SUMMARIZE, LINK, EVICT, DELETE), rather than by updating model weights.

How they measured. On LoCoMo: F1 vs. gpt-4o-mini judge scores under three prompt rubrics (MAGMA, Nemori, SimpleMem), compared by system ranking. Recoverable format-error rate during memory operations, gpt-4o-mini vs. Qwen-2.5-3B. Per-turn retrieval and generation latency, plus offline construction time and token cost. Proposes the Context Saturation Gap = Score_MAG Score_FullContext as a diagnostic.

Findings.

- A weaker open-weight backbone sharply raises format errors during memory operations and lowers answer scores. Evidence: "Nemori: 0.781 answer / 17.91% format error on gpt-4o-mini vs 0.447 / 30.38% on Qwen-2.5-3B; SimpleMem 1.20% vs 4.82%" (Table 4)
- Failed writes silently corrupt long-term memory while short-term conversation stays fluent. Evidence: "while the agent can converse fluently in the short term, its long-term memory becomes corrupted due to failed write operations" (Section 4.4)
- F1 rankings diverge from semantic judge rankings. Evidence: "AMem F1 0.116 (Rank 5) but judge 0.480 (4); SimpleMEM F1 0.268 (Rank 4) but judge 0.294 (5)" (Table 3)
- Lexical metrics reward factually wrong answers that share tokens with the gold answer. Evidence: "Polarity Flip 'not compatible with Mac' F1 0.857, Judge 0.00; Entity Drift F1 0.75, Judge 0.00" (Table 9, Appendix F)
- Many benchmarks fit within modern context windows, so they may not test memory at all. Evidence: "MemBench 100k Tokens, High (Fits in 128k Window); only LongMemEval-M >1M Tokens rated Low (Requires External Memory)" (Table 2)
- Hierarchical paging in MemoryOS makes it impractical for interactive use. Evidence: "MemoryOS Retrieval 31.247s, Total 32.372s per turn vs Full Context 1.726s" (Table 5)
- Offline construction cost varies widely across systems. Evidence: "AMem 15.00 h construction; Nemori 7,044k tokens vs SimpleMem 1,308k" (Table 5, Section 4.5.2)
- Maintenance lag risks memory staleness in structured systems. Evidence: "excessive maintenance overhead risks throughput collapse, where updates lag behind user interactions and memory becomes stale" (Section 4.5.1)

Limitations.

- Only six systems and mainly the LoCoMo dataset. Maintenance cost is discussed qualitatively and omitted from Table 5. Saturation risk is a heuristic, not an empirical test, and is proposed but not reported. Backbone analysis uses two models and two methods. The judge is gpt-4o-mini, the same model family as the controller. Appendix prompt placeholders were left unfilled. Authors note results may vary with implementations, prompts, and API behavior.

Your grades: Capacity _____ Addressing _____ Interference _____ Write distortion _____

1

A12. MultiHop-RAG: MultiHop-RAG: Benchmarking Retrieval-Augmented Generation for Multi-Hop Queries (2024) *Read in full text.* `https://arxiv.org/abs/2401.15391`

Summary. Introduces MultiHop-RAG, a benchmark of 2,556 multi-hop queries (inference, comparison, temporal, null) over 609 news articles (Sep–Dec 2023), with ground-truth evidence sets of 2–4 pieces. Benchmarks embedding retrievers and LLM generators; both retrieval and reasoning fall short on multi-hop queries.

What they did. Standard chunked RAG (LlamaIndex, 256-token chunks, cosine top-K, optional bge-reranker-large over top-20). Queries generated by GPT-4 from claims sharing a bridge-entity/topic, validated by UniEval, manual review and GPT-4 checks.

How they measured. Retrieval: MRR@10, MAP@10, Hits@10, Hits@4 vs ground-truth evidence (null queries excluded). Generation: exact-answer accuracy with retrieved top-6 chunks (2,048 tokens) vs ground-truth evidence chunks.

Findings.

- Even the best retriever+reranker misses much required evidence. Evidence: "the highest Hits@10 is only 0.7467 when the Reranker technique is used. Moreover, the drop in the highest Hits@4 to 0.6625 is worrisome." (Section 4.1 / Table 5)
- Reranking substantially improves retrieval over embedding-only. Evidence: "voyage-02 Hits@10 0.6506 without reranker vs 0.7467 with bge-reranker-large; MRR@10 0.3934 vs 0.586" (Table 5)
- Embedding model choice causes large retrieval variance. Evidence: "jina-embeddings-v2-base-en Hits@10 0.1479 vs bge-large-en-v1.5 0.6718 (without reranker)" (Table 5)
- Retrieval shortfall propagates to generation: GPT-4 accuracy drops from gold to retrieved evidence. Evidence: "GPT-4 0.56 (Retrieved Chunk) vs 0.89 (Ground-truth Chunk); Google-PaLM 0.47 vs 0.74" (Table 6)
- Open-source LLMs fail at multi-hop reasoning even with gold evidence. Evidence: "Llama-2-70b-chat-hf 0.32, Mixtral-8x7B-Instruct 0.36 with ground-truth chunks" (Table 6)
- Error sources for Mixtral include logical negation and temporal ordering; null-query robustness is relatively high. Evidence: "Mixtral-8x7B fails to accurately handle logical negation... often fails to correctly identify the chronological order of events" (Section 4.2 / Figure 3)
- Most queries need 2–4 evidence pieces. Evidence: "2: 42.18%, 3: 30.48%, 4: 15.56%, Null: 11.78%" (Table 4)

Limitations.

- Answers restricted to short forms (yes/no, entity, before/after); max four evidence pieces; only basic LlamaIndex RAG tested (no query decomposition, hybrid retrieval, or agents); queries GPT-4-generated; no breakdown of retrieval metrics by evidence count.

Your grades: Capacity _____ Addressing _____ Interference _____ Write distortion _____

1

A13. H2O: H2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models (2023) *Read in full text.* `https://arxiv.org/abs/2306.14048`

Summary. H2O is a training-free KV cache eviction policy that keeps a fixed budget split between recent tokens and 'heavy hitter' tokens with the highest accumulated attention scores. Evicted KV entries are permanently discarded. The paper motivates the policy with attention sparsity (>95%) and a power-law distribution of accumulated attention. It frames eviction as dynamic submodular maximization and proves a greedy approximation bound. It reports near-full accuracy at a 20% budget and up to 29x throughput gains.

What they did. At each decoding step, the policy sums the normalized attention each cached token has received from preceding queries (a local statistic). It evicts one KV per head so the cache size stays fixed at k. The budget is split evenly between heavy hitters and the most recent tokens, with the recent tokens held in a circular queue. The paper proves a (1-)(1-1/e) greedy guarantee under a submodularity assumption.

How they measured. Accuracy on lm-eval-harness tasks (5-shot) and HELM XSUM/CNN-DM (0-shot) at KV budgets from 4% to 100%. Additional measures: - throughput and latency against FlexGen, DeepSpeed and Accelerate; - perplexity on PG-19 when streaming; - Self-BLEU for output diversity; - ablations comparing heavy hitters only, local tokens only, and both.

Findings.

- Attention in pre-trained LLMs is highly sparse. Evidence: "with a sparsity over 95% in almost all layers" (Section 3.1)
- Masking heavy hitters drastically hurts accuracy. Evidence: "the accuracy drops drastically, confirming the importance of those tokens" (Section 3.2)
- With a 20% budget on OPT-30B, H2O closely matches the full cache, while recency-only eviction collapses. Evidence: "COPA Full 85.00, Local w.o. H2 48.00, Local w. H2 84.00" (Table 2)
- Adding heavy hitters rescues static sparse-attention baselines. Evidence: "Sparse Transformer (strided) w.o. H2 50.00 vs w. H2 83.00 (COPA)" (Table 2)
- Local eviction fails at much higher budgets on summarization. Evidence: "the "Local" strategy collapses at 60% budgets while our H2O can still match the full cache performance with 20% budgets" (Section 5.1 Analysis)
- Neither component alone suffices; heavy hitters contribute more than recent tokens. Evidence: "with a performance degradation from 2.85% to 22.75%" (Section 5.3 Q4)
- Throughput and latency gains over existing systems. Evidence: "up to 3×, 29×, and 29×... reduce the latency by 1.1 1.9× compared to FlexGen" (Section 5.2)
- Low-resource few-shot settings need larger budgets. Evidence: "resulting in a higher KV cache budget (30-40%)" (Appendix C.3)
- Sink-plus-recent streaming (StreamLLM) loses mid-context information; H2O does better. Evidence: "such a strategy will inevitably result in the loss of crucial information" (Appendix C.4)
- Recency-only eviction degenerates the generated text. Evidence: "Self-BELU score of 0.0057 while H2O and the local method achieve 0.0051 and 0.0436" (Appendix C.1)
- H2O is compatible with 4-bit quantization. Evidence: "H2O w. Quant-4bit 84.00 43.20 78.80" (Table 6)

Limitations.

- Accumulated scores bias retention toward early tokens; using averaged scores instead degraded performance. The theoretical guarantee depends on an assumed submodularity of attention. Most tasks are short-sequence multiple-choice; OPT was benchmarked beyond its 2K training length. Evicted tokens cannot be recovered. Parameter memory in the MLP blocks remains unaddressed.

Your grades: Capacity _____ Addressing _____ Interference _____ Write distortion _____

1

A14. EpiKV: Epiphany-Aware KV Cache Eviction Without the Attention Matrix (2026) *Read in abstract only.* `https://arxiv.org/abs/2606.26472`

Summary. Studies attention-based KV eviction for long reasoning traces under tight budgets; finds accuracy loss comes mainly from looping generations that hit the length limit rather than wrong finished answers. Proposes EpiKV, ranking cached tokens via hidden-state shifts plus recent query-key relevance without materializing the attention matrix, compatible with vLLM kernels.

What they did. EpiKV scores cached tokens by combining hidden-state shifts ("epiphany" signal) with the model's recent query-key relevance, obtained from the existing forward pass without the attention matrix, and evicts low-scoring tokens under a fixed budget.

How they measured. Accuracy on multiple reasoning benchmarks vs. strongest attention-based eviction baselines; breakdown of errors into finished-wrong vs. looping to length limit.

Findings.

- With the strongest attention-based eviction, completed generations are not less accurate than full cache. Evidence: "the generations that finish are wrong about as often as without eviction" (abstract)
- Accuracy loss under eviction is dominated by looping. Evidence: "most of the accuracy loss comes from generations that enter loops and run until the length limit" (abstract)
- Keeping more tokens by a fixed importance score worsens looping. Evidence: "retaining more tokens according to a fixed importance score exacerbates this behavior" (abstract)
- Retaining tokens targeted by recent queries prevents looping. Evidence: "What stops the looping is keeping the tokens the model's recent queries point to" (abstract)
- EpiKV matches or beats attention-based baselines while being kernel-compatible. Evidence: "EpiKV matches or outperforms the strongest attention-based eviction baselines while running directly in vLLM with unmodified attention kernels" (abstract)

Limitations.

- Abstract only: no specific budgets, models, benchmarks, or numeric results given.

Your grades: Capacity _____ Addressing _____ Interference _____ Write distortion _____

1

A15. WitCert: WitCert: Sound Runtime Risk Observability and Gating for KV-Cache Quantization (2026) *Read in full text.* `https://arxiv.org/abs/2607.28699`

Summary. WitCert adds a runtime meter to KV-cache quantization. For each (layer, head, decode step) it gives a sound upper bound on the total variation (TV) between exact and compressed attention. Tier A uses per-band residual-norm witnesses and is sound for any cache-preserving black-box quantizer. Tier B is a tighter sub-Gaussian certificate for a subtractively dithered INT8 quantizer. The meter is integrated into SGLang, and gating on it repairs failing schemes: raw fp8 rises from 22.8 to 79.7 on hard RULER tasks. The paper also finds that aggressive schemes survive through cross-layer error cancellation rather than per-step fidelity.

What they did. At write time, the per-band Euclidean norms of the key quantization residual are stored (32 B/tok/head, B=16). Because RoPE is unitary within each band, these norms do not depend on query position. At decode time a per-token logit error bound c_t = (1/d) _b ||q_b|| w_{t,b} is computed (Cauchy–Schwarz) and turned into a TV bound min(1, (A²1)/2) with A = E_p[eˆc]. Tier B instead derives a sub-Gaussian radius from the dither scales, with a request-level budget. The meter is fused into the decode attention kernel and merged across KV splits using the LSE weights. If it exceeds , the top-contributing blocks are paged in from an exact FP16 CPU backing store.

How they measured. - Per-cell true TV compared with the bound (soundness and tightness). - Joint K+V step coverage at , plotted against the request-level budget. - Counts of violations across dither seeds. - RULER and LongBench-E accuracy, with paired bootstrap and McNemar tests. - Teacher-forced top-1/top-5 agreement and KL. - Page-in traffic. - KV capacity and throughput in SGLang.

Findings.

- The prior worst-case certificate (Runtime-Certified Quantized Attention) is too conservative to authorize any compression at practical thresholds. Evidence: "at every practical threshold (0.2) it authorizes zero compression" (Introduction)

- A natural-looking dispersion-based certificate is unsound. It passed aggregate checks and failed when checked per query. Evidence: "The true total variation is 0.049834, while (1) returns 0.006423—a factor of 7.8 ... passed 32,256 checks" (Sec. 3)
- The band-norm witness bound is very loose for low-bit schemes, so it saturates for 1–2 bit quantizers. Evidence: "1289× loose at 4 bits" (Sec. 4.1)
- Under a 1% request-level risk budget, the probabilistic certificate halves the page-in rate compared with the deterministic tanh bound. Evidence: "deterministic bound authorizes 15.14 raw bits with a 44.4% KV-head page-in rate; ours authorizes 11.71 raw bits ... at 22.0%" (Sec. 5)
- Coverage at $=10^2$ ranges widely across models and domains. Evidence: "headline coverage range over the nine model×domain cells is therefore 54.4–81.0% at the single working point =102" (Sec. 5, Table 1)
- No violations were observed under free-running (adaptive) decoding, which lies outside the Tier-B theorem. Evidence: "0/1000 adaptive requests violated ... over 100,352,000 monitored cells ... worst-case TV-to-bound ratio was 0.536" (Sec. 4.2)
- Under YaRN, coverage falls at 128k, while the tanh reference collapses almost completely. A native long-context model (Llama-3.1-8B) shows no such drop. Evidence: "26.6–41.3% at 128k ... tanh reference collapses to at most 0.2% coverage at 128k; Llama-3.1-8B needle 77.0 / 77.1 / 77.0% at 8k/32k/128k" (Sec. 5.2)
- Token eviction collapses silently on needle tasks under the query-agnostic protocol, while 8-bit quantization holds quality. Evidence: "SnapKV tokens 25% mean 55.02, collapse (1.1–65); WitCert quant. bytes 46.5% mean 94.42 vs Full FP16 94.30" (Table 11)
- The meter is discriminative: the cases it sends to fallback have higher error, and the fallback removes most of it. Evidence: "Fallback cases: before →after 9.18×103 →3.32×104 (27.6× reduction)" (Table 13)
- Meter gating at benchmark scale restores the quality floor of broken schemes. Evidence: "fp8 raw 22.8 → fp8 + gate (=5, risk-ranked) 79.7; kivi-2bit raw 73.7 → kivi + gate 79.3; uncompressed 79.3" (Table 18)
- Massive-activation heads drive the meter to very high values. Keeping outlier RoPE pairs in FP16 brings it down. Evidence: "the maximum collapses monotonically: 1.6×10^2 →4.7 →0.086" (Sec. 6.3.7, Table 16)
- Aggressive schemes survive through cross-layer error cancellation, not per-step fidelity. Evidence: "in a 28-layer sweep, no single layer's pollution alone loses anything (0/28); KIVI-2bit distorts 21.9% of steps yet retrieves 95.8%" (abstract; Fig. 6)
- The certified INT8 cache increases serving capacity in SGLang, at a cost in throughput. Evidence: "WitCert int8 1,224,240 (1.88×) ... 511.8 (0.75×) tok/s" (Table 17)
- Stochastic rounding cannot support the certificate, even though it is unbiased. Evidence: "SR's certificate coverage is identically 0.000 at every outlier budget" (Sec. 6.3.6, Table 14)

Limitations.

- - The guarantee is local (per layer, head and step). It does not bound end-task quality. - Tier B is proven only for non-adaptive queries. Adaptive decoding is validated empirically, not proven. - Tier A does not apply to eviction, token merging, latent caches or recurrent-state compression. - The witness is vacuous below about 8 bits. - Coverage is lower on Yi (54–65%) and 72B models (25.5–29.8%). - Most results come from single runs. - In SGLang, the certificate costs 2–16% on short-sequence decoding. - The FlashInfer backend is not covered. Multi-GPU certificate aggregation is not verified. - Non-persistent packed repair pages in 2.81/3.58 GiB per request (P50/P95).

Your grades: Capacity _____ Addressing _____ Interference _____ Write distortion _____

---

4. Part B: screening check

1

S1. OSDN: Improving Delta Rule with Provable Online Preconditioning in Linear Attention (2026) Linear attention and state-space models offer constant-memory alternatives to softmax attention, but often struggle with in-context associative recall. The Delta Rule mitigates this by writing each token via one step of online gradient descent. However, its step size relies on a single scalar gate that ignores the feature-wise curvature of the inner objective. We propose Online Scaled DeltaNet (OSDN), which augments the scalar gate with a diagonal preconditioner updated online via hypergradient feedback. Crucially, this right-preconditioning is algebraically equivalent to a per-feature scaling of the write-side key. This equivalence allows OSDN to strictly preserve the hardware-friendly chunkwise parallel pipeline of DeltaNet without incurring high-dimensional state overhead. Theoretically, by exploiting the exact-quadratic structure of the inner regression loss, we establish super-geometric convergence against a right-Newton comparator and prove an algorithm-aligned token-local residual contraction bound. To handle non-stationary contexts, we further introduce Adaptive Preconditioner Forgetting (APF) to dynamically refresh stale calibration. Empirically, OSDN demonstrates strong performance across scales. At the 340M-parameter scale, OSDN improves JRT-style in-context recall by 32% over DeltaNet. Scaling to 1.3B parameters, it achieves a 39% reduction in the recall residual ratio while maintaining parity on general downstream tasks (e.g., perplexity and LongBench) – demonst

Relevant? yes / no

1

S2. Retrieval Augmented Generation for the Detection of Major Bleeding Events in the Electronic Health Record (2024) Background: Non-Surgical Major Bleeding (NSMB) is an important clinical event associated with significant morbidity and mortality. Detecting NSMB events in the electronic health record is central to conducting research and quality assessment studies for patients on anticoagulation therapy and in other settings as well. Determining a patient's NSMB history by means of manual clinical chart review is limited by time and human resources concerns. Natural language processing (NLP) has emerged as a promising modality for the detection of clinical events in the electronic health record. Transformer NLP models have revolutionized the field and have performed satisfactorily for varied clinical endpoints, including venous thromboembolism for which this approach has been extensively validated. However, there is a lack of data concerning the use of NLP to detect bleeding events. Training such models generally requires an arbitrarily large, annotated dataset, which necessitates significant amounts of human labor. The use of generative large language models (LLM's) with retrieval augmented generation (RAG) is emerging as a promising modality to generate machine-derived labels without a large human-annotated set. RAG seeks to avoid hallucinations and incomplete knowledge in LLM's by providing domain-specific context from external sources. We hypothesized RAG could be used to detect NSMB events in cancer patients. Methods: Clinical notes for 174 patients seen at Memorial Sloan Kettering Can

Relevant? yes / no

1

S3. Automated Summarization of Financial News Using Large Language Models and Retrieval-Augmented Generation: An Early Empirical Study (Fall 2023) (2026) Stock market analysts and investors face a daily challenge: too much financial news, too little time. Manually reading and synthesizing hundreds of company-specific articles is impractical, yet missing key information can directly affect investment decisions. This project, conducted at George Washington University in Fall 2023, explores whether Large Language Models can automate this process reliably. We built a pipeline that pulls news articles from the News API, company background from Wikipedia, and stock price data from Yahoo Finance for ten major companies (AAPL, MSFT, GOOGL, AMZN, META, TSLA, JPM, NVDA, WMT, DIS). Because LLMs cannot directly process numerical tables, we developed a simple but effective template that converts stock data into natural language narratives. We then tested two summarization approaches (Summarize Chains and Retrieval-Augmented Generation with FAISS) across three open-source models (Falcon-7B-Instruct, DistilBART-CNN-12-6, BART-Large-XSum) for news, and GPT (text-davinci-003) for stock summaries. Falcon-7B with Summarize Chains gave the best results, covering all news events accurately and coherently. RAG, while promising in theory, caused severe repetition in Falcon and hallucinated facts in BART-Large when k was large. Both LLM-based approaches outperformed a simple

Lead-3 baseline on ROUGE-1. We also built a Streamlit dashboard for interactive stock visualization. The work was done in Fall 2023, before RAG-based financial tools became wides

Relevant? yes / no

1

S4. ActionCLIP: A New Paradigm for Video Action Recognition (2021) The canonical approach to video action recognition dictates a neural model to do a classic and standard 1-of-N majority vote task. They are trained to predict a fixed set of predefined categories, limiting their transferable ability on new datasets with unseen concepts. In this paper, we provide a new perspective on action recognition by attaching importance to the semantic information of label texts rather than simply mapping them into numbers. Specifically, we model this task as a video-text matching problem within a multimodal learning framework, which strengthens the video representation with more semantic language supervision and enables our model to do zero-shot action recognition without any further labeled data or parameters requirements. Moreover, to handle the deficiency of label texts and make use of tremendous web data, we propose a new paradigm based on this multimodal learning framework for action recognition, which we dub "pre-train, prompt and fine-tune". This paradigm first learns powerful representations from pre-training on a large amount of web image-text or video-text data. Then it makes the action recognition task to act more like pre-training problems via prompt engineering. Finally, it end-to-end fine-tunes on target datasets to obtain strong performance. We give an instantiation of the new paradigm, ActionCLIP, which not only has superior and flexible zero-shot/few-shot transfer ability but also reaches a top performance on general action recognition

Relevant? yes / no

1

S5. 'Redeeming the body': Embodiment and the 'other' in the work of Marion Milner (2021) it may be that this disinclination to use technical terms has helped me to get a better hold on what seems to have been my deepest preoccupation over the years: that is to do with one's sense of being alive and inhabiting one's body, what I have called one's body presentation, as against body representations or body images, this sense of the inner dark matrix from which emerges drives to action or thoughts or emotional expression or new perceivings.11 Marion Milner, '1986: Afterthoughts', in The Suppressed Madness of Sane Men: Forty-Four Years of Exploring Psychoanalysis (Hove: Brunner-Routledge, 2005), 234–42 (p.235). There is something antithetical, Milner seems to say, between psychoanalytic terminology and understanding what I will call the 'lived body', a central line of thinking in her oeuvre. While Milner's formulation emphasises what her 'disinclination to use technical terms' has allowed her to do – thereby keeping the focus on her own thought – another way of reading this passage is that she has identified a constraint within psychoanalytic discourse, which has in some way not been conducive to thinking about bodily experience. Perhaps Milner's modesty here obscures what is one of her most radical contributions to psychoanalysis: a deep engagement with embodied experience and its role in the intersubjective relationship between self and other, analyst and analysand. In her view, attending to one's body from the inside – deepening one's awareness of bodily experience

Relevant? yes / no

1

S6. Temporal Window Smoothing of Exogenous Variables for Improved Time Series Prediction (2025) Although most transformer-based time series forecasting models primarily depend on endogenous inputs, recent state-of-the-art approaches have significantly improved performance by incorporating external information through exogenous inputs. However, these methods face challenges, such as redundancy when endogenous and exogenous inputs originate from the same source and limited ability to capture long-term dependencies due to fixed look-back windows. In this paper, we propose a method that whitens the exogenous input to reduce redundancy that may persist within the data based on global statistics. Additionally, our approach helps the exogenous input to be more aware of patterns and trends over extended periods. By introducing this refined, globally context-aware exogenous input to the endogenous input without increasing the lookback window length, our approach guides the model towards improved forecasting. Our approach achieves state-of-the-art performance in four benchmark datasets, consistently outperforming 11 baseline models. These results establish our method as a robust and effective alternative for using exogenous inputs in time series forecasting.

Relevant? yes / no

1

S7. Nonparametric Masked Language Modeling (2023) Existing language models (LMs) predict tokens with a softmax over a finite vocabulary, which can make it difficult to predict rare tokens or phrases.We introduce NPM, the first nonparametric masked language model that replaces this softmax with a nonparametric distribution over every phrase in a reference corpus.NPM fills in the [MASK] solely from retrieving a token from a text corpus.We show that NPM can be efficiently trained with a contrastive objective and an in-batch approximation to full corpus retrieval.Zero-shot evaluation on 16 tasks including classification, fact probing and question answering demonstrates that NPM outperforms significantly larger parametric models, either with or without a retrieve-and-generate approach.It is particularly better at dealing with rare patterns (word senses or facts) and predicting rare or nearly unseen words (e.g., non-Latin script).We release the model and code at github.com/facebookresearch/NPM.

Relevant? yes / no

1

S8. QoKV: Comprehending and Surpassing the Hurdles of KV Cache Quantization (2025) Large language models (LLMs) have demonstrated outstanding performance in various tasks. However, the memory footprint of the key-value (KV) cache generated during model inference poses significant challenges for efficient model deployment. This paper presents a detailed analysis of the KV cache and identifies potential sources of quantization errors. We find that the attention scores exhibit a strong power-law distribution while showing an additional attention to the initial and recent tokens, and that only a small subset of outlier channels in KV cache span a wide numerical range. Based on these findings, we propose QoKV, an efficient post-training quantization (PTQ) method designed for the KV cache, which facilitates accurate low-bit quantization through a novel group evaluation strategy and numerical scaling. Extensive experimental results across various tasks demonstrate that our proposed method outperforms existing methods, achieving near-floating-point performance while saving 73.75% of the cache footprint.

Relevant? yes / no

1

S9. Design and Evaluation of a Retrieval-Augmented Generation Architecture for OWASP Security Data (2023) Project Description This archive contains a research artifact for the design, implementation, and evaluation of a Retrieval-Augmented Generation (RAG) system in the domain of application security. The artifact provides a complete, self-contained snapshot of the system architecture, curated evaluation data, and automated benchmarking pipelines. RAG System Architecture The archive includes a fully implemented RAG pipeline designed to ground large language model outputs in authoritative security documentation. The system follows a modular architecture with clearly separated components for document ingestion, semantic retrieval, and answer generation. The knowledge base is derived from OWASP Cheat Sheet Series and related security guidance, enabling security-focused question answering with reduced risk of hallucinated responses. Ground Truth Dataset A ground truth dataset is included to support systematic evaluation. The dataset consists of security-relevant questions and corresponding reference information derived from OWASP materials. It is intended exclusively for offline evaluation and benchmarking of RAG behavior and is not meant to serve as training data.

Relevant? yes / no

1

S10. Generative AI as External Operational Mediation: Open Research Materials, Supplementary Methodological Guide, Figures, and Coded Corpus Dataset (v15) (2026) This Zenodo record provides the open-access research materials, supplementary methodological documentation, figure package, and coded corpus dataset associated with the manuscript: “Generative AI as External Operational Mediation: A Conceptual Integrative Review of Cognitive Externalization and Human-AI Division of Labor” Author: Roman R. Sidorchuk Affiliations: Plekhanov Russian University of Economics; RUDN University ORCID: 0000-0002-4033-2937 Correspondence: Sidorchuk.RR@rea.ru Version: v15 DOI: 10.5281/zenodo.20026337 Publication date of this record: 2026-05-04 License: Creative Commons Attribution 4.0 International (CC BY 4.0) Copyright: © 2026 Roman R. Sidorchuk

This record is an open research materials package prepared to support transparency, reproducibility, independent inspection, and citation of the conceptual integrative review. The package is not a substitute for the journal submission itself and should be understood as an open-access research object containing the author manuscript version, supplementary methodological documentation, coded literature corpus, figures, metadata files, and reproducibility-oriented materials. The manuscript conceptualizes generative artificial intelligence (GenAI) as a form of external operational mediation within human-AI cognitive systems. The central argument is that GenAI should not be reduced to text generation, productivity enhancement, or digital writing. Instead, the review frames GenAI as an external computational environm

Relevant? yes / no

1

S11. Q-KVComm: Efficient Multi-Agent Communication via Adaptive KV Cache Compression (2026) Multi-agent Large Language Model (LLM) systems face a critical bottleneck: redundant transmission of contextual information between agents consumes excessive bandwidth and computational resources. Traditional approaches discard internal semantic representations and transmit raw text, forcing receiving agents to recompute similar representations from scratch. We introduce Q-KVComm, a new protocol that enables direct transmission of compressed key-value (KV) cache representations between LLM agents. Q-KVComm combines three key innovations: (1) adaptive layer-wise quantization that allocates variable bit-widths based on sensitivity profiling, (2) hybrid information extraction that preserves critical facts across content domains, and (3) heterogeneous model calibration establishing cross-architecture communication. Extensive experiments across three diverse question-answering datasets demonstrate that QKVComm achieves 5-6x compression ratios while maintaining semantic fidelity, with coherence quality scores above 0.77 across all scenarios. The protocol exhibits robust performance across model sizes (1.1B-1.5B parameters) and adapts to real-world applications including conversational QA and multi-hop reasoning. Our work establishes a new paradigm for LLM agent communication, shifting from text-based to representation-based information exchange.

Relevant? yes / no

1

S12. Merging Generated and Retrieved Knowledge for Open-Domain QA (2023) Open-domain question answering (QA) systems are often built with retrieval modules.However, retrieving passages from a given source is known to suffer from insufficient knowledge coverage.Alternatively, prompting large language models (LLMs) to generate contextual passages based on their parametric knowledge has been shown to improve QA performance.Yet, LLMs tend to "hallucinate" content that conflicts with the retrieved knowledge.Based on the intuition that answers supported by both sources are more likely to be correct, we propose COMBO, a Compatibility-Oriented knowledge Merging for Better Open-domain QA framework, to effectively leverage the two sources of information.Concretely, we match LLM-generated passages with retrieved counterparts into compatible pairs, based on discriminators trained with silver compatibility labels.Then a Fusionin-Decoder-based (Izacard and Grave, 2021b) reader model handles passage pairs to arrive at the final answer.Experiments show that COMBO outperforms competitive baselines on three out of four tested open-domain QA benchmarks.Further analysis reveals that our proposed framework demonstrates greater efficacy in scenarios with a higher degree of knowledge conflicts.

Relevant? yes / no

1

S13. Numerical Weather Forecasting Using Convolutional-LSTM With Attention and Context Matcher Mechanisms (2024) Numerical weather forecasting with high-resolution physical models requires extensive computational resources on supercomputers, often making it impractical for real-life applications. Alternatively, deep learning methods can provide results within minutes of receiving data. Although baseline deep learning models can make accurate short-term predictions, their performance deteriorates rapidly as the output sequence length increases. However, many real-life scenarios require long-term prediction of certain weather features to mitigate and take advantage of the effects of high-impact weather events. In response, we introduce the Weather Model, which provides rapid and accurate long-term spatial predictions for high-resolution spatio-temporal weather data. We integrate a stacked ConvLSTM network as our building block, given its accuracy in capturing spatial data patterns through convolution operations. Furthermore, we additionally incorporate attention and context

matcher mechanisms. The attention mechanism allows the effective usage of the side-information vector by selectively focusing on different parts of the input sequence at each time step. Concurrently, the context matcher mechanism enhances the network's ability to preserve long-term dependencies. Our Weather Model achieves significant performance improvements compared to baseline deep learning models, including ConvLSTM, TrajGRU and U-Net. Our experimental evaluation involves high-scale, real-world benchmark numerical w

Relevant? yes / no

1

S14. Escaping the Context Bottleneck: Active Context Curation for LLM Agents via Reinforcement Learning (2026) Large Language Models (LLMs) struggle with long-horizon tasks due to the "context bottleneck" and the "lost-in-the-middle" phenomenon, where accumulated noise from verbose environments degrades reasoning over multi-turn interactions. To address this issue, we introduce a symbiotic framework that decouples context management from task execution. Our architecture pairs a lightweight, specialized policy model, ContextCurator, with a powerful frozen foundation model, TaskExecutor. Trained via reinforcement learning, ContextCurator actively reduces information entropy in the working memory. It aggressively prunes environmental noise while preserving reasoning anchors, that is, sparse data points that are critical for future deductions. On WebArena, our framework improves the success rate of Gemini-3.0-flash from 36.4% to 41.2% while reducing token consumption by 8.8% (from 47.4K to 43.3K). On DeepSearch, it achieves a 57.1% success rate, compared with 53.9%, while reducing token consumption by a factor of 8. Remarkably, a 7B ContextCurator matches the context management performance of GPT-4o, providing a scalable and computationally efficient paradigm for autonomous long-horizon agents.

Relevant? yes / no

1

S15. Multi-Task Learning With Multi-Query Transformer for Dense Prediction (2023) Previous multi-task dense prediction studies developed complex pipelines such as multi-modal distillations in multiple stages or searching for task relational contexts for each task. The core insight beyond these methods is to maximize the mutual effects of each task. Inspired by the recent query-based Transformers, we propose a simple pipeline named Multi-Query Transformer (MQTransformer) that is equipped with multiple queries from different tasks to facilitate the reasoning among multiple tasks and simplify the cross-task interaction pipeline. Instead of modeling the dense per-pixel context among different tasks, we seek a task-specific proxy to perform cross-task reasoning via multiple queries where each query encodes the task-related context. The MQTransformer is composed of three key components: shared encoder, cross-task query attention module and shared decoder. We first model each task with a task-relevant query. Then both the task-specific feature output by the feature extractor and the task-relevant query are fed into the shared encoder, thus encoding the task-relevant query from the task-specific feature. Secondly, we design a cross-task query attention module to reason the dependencies among multiple task-relevant queries; this enables the module to only focus on the query-level interaction. Finally, we use a shared decoder to gradually refine the image features with the reasoned query features from different tasks. Extensive experiment results on two dense predic

Relevant? yes / no

1

S16. Bridging the Gap Between Anchor-Based and Anchor-Free Detection via Adaptive Training Sample Selection (2020) Object detection has been dominated by anchor-based detectors for several years. Recently, anchor-free detectors have become popular due to the proposal of FPN and Focal Loss. In this paper, we first point out that the essential difference between anchor-based and anchor-free detection is actually how to define positive and negative training samples, which leads to the performance gap between them. If they adopt the same definition of positive and negative samples during training, there is no obvious difference in the final performance, no matter regressing from a box or a point. This shows that how to select positive and negative training samples is important for current object detectors. Then, we propose an Adaptive Training Sample Selection (ATSS) to automatically select positive and negative samples according to statistical characteristics of object. It significantly improves the performance of anchor-based and anchor-free detectors and bridges the gap between them. Finally, we discuss the necessity of tiling multiple anchors per location on the image to detect objects. Extensive experiments conducted

on MS COCO support our aforementioned analysis and conclusions. With the newly introduced ATSS, we improve state-of-the-art detectors by a large margin to 50.7% AP without introducing any overhead. The code is available at https://github.com/sfzhang15/ATSS.

Relevant? yes / no

1

S17. Benchmarking Large Language Models in Retrieval-Augmented Generation (2023) Retrieval-Augmented Generation (RAG) is a promising approach for mitigating the hallucination of large language models (LLMs). However, existing research lacks rigorous evaluation of the impact of retrieval-augmented generation on different large language models, which make it challenging to identify the potential bottlenecks in the capabilities of RAG for different LLMs. In this paper, we systematically investigate the impact of Retrieval-Augmented Generation on large language models. We analyze the performance of different large language models in 4 fundamental abilities required for RAG, including noise robustness, negative rejection, information integration, and counterfactual robustness. To this end, we establish Retrieval-Augmented Generation Benchmark (RGB), a new corpus for RAG evaluation in both English and Chinese. RGB divides the instances within the benchmark into 4 separate testbeds based on the aforementioned fundamental abilities required to resolve the case. Then we evaluate 6 representative LLMs on RGB to diagnose the challenges of current LLMs when applying RAG. Evaluation reveals that while LLMs exhibit a certain degree of noise robustness, they still struggle significantly in terms of negative rejection, information integration, and dealing with false information. The aforementioned assessment outcomes indicate that there is still a considerable journey ahead to effectively apply RAG to LLMs.

Relevant? yes / no

1

S18. Exploiting a Joint Embedding Space for Generalized Zero-Shot Semantic Segmentation (2021) We address the problem of generalized zero-shot semantic segmentation (GZS3) predicting pixel-wise semantic labels for seen and unseen classes. Most GZS3 methods adopt a generative approach that synthesizes visual features of unseen classes from corresponding semantic ones (e.g., word2vec) to train novel classifiers for both seen and unseen classes. Although generative methods show decent performance, they have two limitations: (1) the visual features are biased towards seen classes; (2) the classifier should be retrained whenever novel unseen classes appear. We propose a discriminative approach to address these limitations in a unified framework. To this end, we leverage visual and semantic encoders to learn a joint embedding space, where the semantic encoder transforms semantic features to semantic prototypes that act as centers for visual features of corresponding classes. Specifically, we introduce boundary-aware regression (BAR) and semantic consistency (SC) losses to learn discriminative features. Our approach to exploiting the joint embedding space, together with BAR and SC terms, alleviates the seen bias problem. At test time, we avoid the retraining process by exploiting semantic prototypes as a nearest-neighbor (NN) classifier. To further alleviate the bias problem, we also propose an inference technique, dubbed Apollonius calibration (AC), that modulates the decision boundary of the NN classifier to the Apollonius circle adaptively. Experimental results demonstrate

Relevant? yes / no

1

S19. WeightedKV: Attention Scores Weighted Key-Value Cache Merging for Large Language Models (2025) Large Language Models (LLMs) use key-value (KV) cache to reduce redundant computation in autoregressive generation. However, the KV cache size increases linearly during generation, leading to excessive memory usage, especially for long texts. Most KV cache compression methods evict the unimportant KV pairs to maintain a fixed cache size, which leads to the permanent loss of tokens during generation. However, singular value decomposition shows that values do not exhibit a strong low-rank property as keys do, suggesting that information is distributed more evenly across values, in contrast to its more redundant distribution within keys. Therefore, methods that evict both keys and values risk losing crucial information and compromise context integrity, ultimately degrading the output quality. To address this problem, we propose WeightedKV, a novel, training-free approach that discards the keys of less important tokens, while merging their values into neighboring tokens via a convex combination weighted by their average attention scores. In this way, the retained keys serve as anchors that guide the generation process, while the merged values provide a rich contextual backdrop. We assess our method

on four widely used language modeling datasets, demonstrating superior performance compared to all baseline methods, particularly with a lower budget ratio.

Relevant? yes / no

1

S20. AudioKV: KV Cache Eviction in Efficient Large Audio Language Models (2026) Large Audio-Language Models (LALMs) have set new benchmarks in speech processing, yet their deployment is hindered by the memory footprint of the Key-Value (KV) cache during long-context inference. While general KV cache compression techniques excel in LLMs, they often fail in the audio domain by overlooking the intrinsic temporal continuity of acoustic signals. To bridge this gap, we propose AudioKV, a novel framework that robustly prioritizes audio-critical attention heads through a hardware-friendly semantic-acoustic alignment mechanism. Specifically, we identify these modality-specialized heads by analyzing attention scores in ASR tasks and dynamically allocate KV cache budgets preferentially to them. Furthermore, we introduce Spectral Score Smoothing (SSS), an FFT-based global filtering strategy designed to suppress high-frequency noise and recover smooth global trends from importance scores, ensuring more balanced token selection with unprecedented precision. Extensive evaluations across multiple LALMs, including Qwen and Gemma series, demonstrate that AudioKV significantly outperforms baselines while enhancing computational efficiency. Notably, at a 40% compression ratio, AudioKV maintains near-full accuracy on Qwen3-Omni-30B with only a 0.45% drop, whereas traditional methods suffer from catastrophic performance degradation and repetition. Our code will be released after acceptance.

Relevant? yes / no

1

S21. Quant VideoGen: Auto-Regressive Long Video Generation via 2-Bit KV-Cache Quantization (2026) Despite rapid progress in autoregressive video diffusion, an emerging system algorithm bottleneck limits both deployability and generation capability: KV cache memory. In autoregressive video generation models, the KV cache grows with generation history and quickly dominates GPU memory, often exceeding 30 GB, preventing deployment on widely available hardware. More critically, constrained KV cache budgets restrict the effective working memory, directly degrading long horizon consistency in identity, layout, and motion. To address this challenge, we present Quant VideoGen (QVG), a training free KV cache quantization framework for autoregressive video diffusion models. QVG leverages video spatiotemporal redundancy through Semantic Aware Smoothing, producing low magnitude, quantization friendly residuals. It further introduces Progressive Residual Quantization, a coarse to fine multi stage scheme that reduces quantization error while enabling a smooth quality memory trade off. Across LongCat Video, HY WorldPlay, and Self Forcing benchmarks, QVG establishes a new Pareto frontier between quality and memory efficiency, reducing KV cache memory by up to 7.0 times with less than 4% end to end latency overhead while consistently outperforming existing baselines in generation quality. Code is available at: https://github.com/svg-project/Quant-VideoGen

Relevant? yes / no

1

S22. Retrieval-Based Prompt Selection for Code-Related Few-Shot Learning (2023) Large language models trained on massive code corpora can generalize to new tasks without the need for task-specific fine-tuning. In few-shot learning, these models take as input a prompt, composed of natural language instructions, a few instances of task demonstration, and a query and generate an output. However, the creation of an effective prompt for code-related tasks in few-shot learning has received little attention. We present a technique for prompt creation that automatically retrieves code demonstrations similar to the developer task, based on embedding or frequency analysis. We apply our approach, Cedar, to two different programming languages, statically and dynamically typed, and two different tasks, namely, test assertion generation and program repair. For each task, we compare Cedar with state-of-the-art task-specific and fine-tuned models. The empirical results show that, with only a few relevant code demonstrations, our prompt creation technique is effective in both tasks with an accuracy of 76% and 52% for exact matches in test assertion generation and program repair tasks, respectively. For assertion generation, Cedar outperforms existing task-specific and fine-tuned models by 333% and 11%, respectively. For program repair, Cedar yields 189% better accuracy than task-specific models and is competitive with recent

fine-tuned models. These findings have practical implications for practitioners, as Cedar could potentially be applied to multilingual and multitask

Relevant? yes / no

1

S23. MemInsight: Autonomous Memory Augmentation for LLM Agents (2025) Large language model (LLM) agents have evolved to intelligently process information, make decisions, and interact with users or tools. A key capability is the integration of long-term memory capabilities, enabling these agents to draw upon historical interactions and knowledge. However, the growing memory size and need for semantic structuring pose significant challenges. In this work, we propose an autonomous memory augmentation approach, MemInsight, to enhance semantic data representation and retrieval mechanisms. By leveraging autonomous augmentation to historical interactions, LLM agents are shown to deliver more accurate and contextualized responses. We empirically validate the efficacy of our proposed approach in three task scenarios; conversational recommendation, question answering and event summarization. On the LLM-REDIAL dataset, MemInsight boosts persuasiveness of recommendations by up to 14%. Moreover, it outperforms a RAG baseline by 34% in recall for LoCoMo retrieval. Our empirical results show the potential of MemInsight to enhance the contextual performance of LLM agents across multiple tasks.

Relevant? yes / no

1

S24. Towards Automated Smart Contract Generation: Evaluation, Benchmarking, and Retrieval-Augmented Repair (2025) Smart contracts, predominantly written in Solidity and deployed on blockchains such as Ethereum, are immutable after deployment, making functional correctness critical. However, existing evaluations of Solidity code generation rely largely on surface-level metrics (e.g., BLEU, CrystalBLEU) or manual inspection, which correlate poorly with functional correctness. In contrast to Python, Solidity lacks large-scale, execution-based benchmarks, limiting systematic evaluation of large language models for smart contract development. We introduce SolBench, a comprehensive benchmark and automated testing pipeline for Solidity that emphasizes functional correctness via differential fuzzing. SolBench consists of 28825 functions extracted from 7604 real-world smart contracts collected from Etherscan (genesis-2024), spanning ten application domains. We benchmark 14 diverse LLMs, covering open and closed models, 1.3B-671B parameters, and both general-purpose and code-specialized architectures. The dominant failure mode is missing critical intra-contract information, such as state variables and type definitions. Providing full-contract context improves accuracy but incurs prohibitive inference costs. To address this, we propose Retrieval-Augmented Repair (RAR), a cost-effective framework that integrates execution feedback into code repair. RAR uses compiler and runtime error messages to retrieve only the minimal contract snippets needed to correct a target function, avoiding full-context in

Relevant? yes / no

1

S25. Advancing Retrieval-Augmented Generation with Inverted Question Matching for Enhanced QA Performance (2024) This work presents a novel architecture for building Retrieval-Augmented Generation (RAG) systems to improve Question Answering (QA) tasks from a target corpus. Large Language Models (LLMs) have revolutionized the analyzing and generation of human-like text. These models rely on pre-trained data and lack real-time updates unless integrated with live data tools. RAG enhances LLMs by integrating online resources and databases to generate contextually appropriate responses. However, traditional RAG still encounters challenges like information dilution and hallucinations when handling vast amounts of data. Our approach addresses these challenges by converting corpora into a domain-specific dataset and RAG architecture is constructed to generate responses from the target document. We introduce QuIM-RAG (Question-to-question Inverted Index Matching), a novel approach for the retrieval mechanism in our system. This strategy generates potential questions from document chunks and matches these with user queries to identify the most relevant text chunks for generating accurate answers. We have implemented our RAG system on top of the open-source Meta-LLaMA3-8B-instruct model by Meta Inc. that is available on Hugging Face. We constructed a custom corpus of 500+ pages from a high-traffic website accessed thousands of times daily for answering complex questions, along with manually

prepared ground truth QA for evaluation. We compared our approach with traditional RAG models using BERT-Sco

Relevant? yes / no

1

S26. KeyDiff: Key Similarity-Based KV Cache Eviction for Long-Context LLM Inference in Resource-Constrained Environments (2025) We demonstrate that geometrically distinctive keys during LLM inference tend to have high attention scores. Based on the phenomenon we propose KeyDiff, a training-free KV cache eviction method based solely on key similarity. Unlike other KV cache eviction methods, KeyDiff can process arbitrarily long prompts within strict resource constraints and efficiently generate responses. We provide a theoretical basis for KeyDiff by relating key diversity with attention scores. These results imply KeyDiff can efficiently identify the most important tokens to retain. Notably KeyDiff does not rely on attention scores, allowing the use of optimized attention mechanisms like FlashAttention. Under a strict memory allowance, we demonstrate the effectiveness of KeyDiff for the Llama and Qwen model families by observing a performance gap of less than 0.04% with 8K cache budget ($\{}sim$23% KV cache reduction) from the non-evicting baseline on LongBench for Llama 3.1-8B and Llama 3.2-3B. We also observe near baseline performance for Deepseek-R1-Distill-Llama-8B on the Math500 reasoning benchmark and decrease end-to-end inference latency by up to 30% compared to the other token-eviction methods.

Relevant? yes / no

1

S27. FCOS: Fully Convolutional One-Stage Object Detection (2019) We propose a fully convolutional one-stage object detector (FCOS) to solve object detection in a per-pixel prediction fashion, analogue to semantic segmentation. Almost all state-of-the-art object detectors such as RetinaNet, SSD, YOLOv3, and Faster R-CNN rely on pre-defined anchor boxes. In contrast, our proposed detector FCOS is anchor box free, as well as proposal free. By eliminating the pre-defined set of anchor boxes, FCOS completely avoids the complicated computation related to anchor boxes such as calculating overlapping during training. More importantly, we also avoid all hyper-parameters related to anchor boxes, which are often very sensitive to the final detection performance. With the only post-processing non-maximum suppression (NMS), FCOS with ResNeXt-64x4d-101 achieves 44.7% in AP with single-model and single-scale testing, surpassing previous one-stage detectors with the advantage of being much simpler. For the first time, we demonstrate a much simpler and flexible detection framework achieving improved detection accuracy. We hope that the proposed FCOS framework can serve as a simple and strong alternative for many other instance-level tasks. Code is available at: https://tinyurl.com/FCOSv1.

Relevant? yes / no

1

S28. Integrated Multimodal AI System for Retrieval-Augmented Reasoning, Object Sensing, and Damage Analysis (2026) This work presents a unified multimodal AI system for damage assessment that integrates retrieval-augmented generation (RAG) models, thermal spectrum perception, vision foundation model pipelines, and exploratory wireless signal sensing. A RAG component is developed to ground a locally hosted language model in project-specific documentation, including specialized damage level classification criteria to mitigate hallucinations during inference. Controlled comparisons against static few-shot prompting demonstrate that dynamic retrieval improves grounding and factual consistency. We further compare vector-based RAG with a knowledge graph variant constructed via entity-relation extraction, and show that graph-based retrieval produces stronger responses for damage assessment queries requiring cross-document reasoning, motivating hybrid dense, sparse, and graph-aware retrieval. To address limitations of EO imagery under adverse lighting and weather conditions, infrared (IR)/thermal sensing is employed for object detection and segmentation. Our detectors generate candidate detections, yielding improved segmentation of a broad array of objects. Paired IR versus visible spectrum tracking experiments reveal failure modes, motivating multimodal fusion for robust object detection and damage analysis. Vision foundation and vision-language models are leveraged to generate synthetic damage imagery and classify damage severity with high accuracy, supporting training and validation of downstr

Relevant? yes / no

1

S29. XQuant: Achieving Ultra-Low Bit KV Cache Quantization with Cross-Layer Compression (2025) Large Language Models (LLMs) have demonstrated remarkable capabilities across diverse natural language processing tasks. However, their extensive memory requirements, particularly due to KV cache growth during long-text understanding and generation, present significant challenges for deployment in resource-constrained environments. Quantization has emerged as a promising solution to reduce memory consumption while preserving historical information. We propose XQuant, a training-free and plug-and-play framework that achieves ultra-low equivalent bit-width KV cache quantization. XQuant introduces two key innovations: a computationally negligible data-free calibration method and cross-layer KV cache compression, enabling quantization to sub-1.4 bits. Extensive experiments on TruthfulQA and LongBench demonstrate that XQuant outperforms state-of-the-art methods (e.g., KIVI-2bit and AsymKV-1.5bit) by achieving lower bit-width while maintaining superior performance, establishing a better trade-off between memory efficiency and model accuracy.

Relevant? yes / no

1

S30. Agent Drift: Quantifying Behavioral Degradation in Multi-Agent LLM Systems Over Extended Interactions (2026) Multi-agent Large Language Model (LLM) systems have emerged as powerful architectures for complex task decomposition and collaborative problem-solving. However, their long-term behavioral stability remains largely unexamined. This study introduces the concept of agent drift, defined as the progressive degradation of agent behavior, decision quality, and inter-agent coherence over extended interaction sequences. We present a comprehensive theoretical framework for understanding drift phenomena, proposing three distinct manifestations: semantic drift (progressive deviation from original intent), coordination drift (breakdown in multi-agent consensus mechanisms), and behavioral drift (emergence of unintended strategies). We introduce the Agent Stability Index (ASI), a novel composite metric framework for quantifying drift across twelve dimensions, including response consistency, tool usage patterns, reasoning pathway stability, and inter-agent agreement rates. Through simulation-based analysis and theoretical modeling, we demonstrate how unchecked agent drift can lead to substantial reductions in task completion accuracy and increased human intervention requirements. We propose three mitigation strategies: episodic memory consolidation, drift-aware routing protocols, and adaptive behavioral anchoring. Theoretical analysis suggests these approaches can significantly reduce drift-related errors while maintaining system throughput. This work establishes a foundational methodology f

Relevant? yes / no

1

S31. Cross-layer Attention Sharing for Pre-trained Large Language Models (2024) To enhance the efficiency of the attention mechanism within large language models (LLMs), previous works primarily compress the KV cache or group attention heads, while largely overlooking redundancy between layers. Our comprehensive analyses across various LLMs show that highly similar attention patterns persist within most layers. It's intuitive to reduce the redundancy by sharing attention weights across layers. However, further analysis reveals two challenges: (1) Directly sharing the weight matrix without carefully rearranging the attention heads proves to be ineffective; (2) Shallow layers are vulnerable to small deviations in attention weights. Driven by these insights, we introduce LISA, a lightweight substitute for self-attention in well-trained LLMs. LISA employs tiny feed-forward networks to align attention heads between adjacent layers and low-rank matrices to approximate differences in layer-wise attention weights. Evaluations encompassing 13 typical benchmarks demonstrate that LISA maintains high response quality in terms of accuracy and perplexity while reducing redundant attention calculations within 53%-84% of the total layers. Our implementations of LISA achieve a 6x compression of Q and K matrices within the attention mechanism, with maximum throughput improvements 19.5%, 32.3%, and 40.1% for LLaMA3-8B, LLaMA2-7B, and LLaMA2-13B, respectively.

Relevant? yes / no

1

S32. Never Lost in the Middle: Mastering Long-Context Question Answering with Position-Agnostic Decompositional Training (2023) While large language models (LLMs) are equipped with longer text input capabilities

than before, they are struggling to seek correct information in long contexts. The "lost in the middle" problem challenges most LLMs, referring to the dramatic decline in accuracy when correct information is located in the middle. To overcome this crucial issue, this paper proposes to enhance the information searching and reflection ability of LLMs in long contexts via specially designed tasks called Attention Strengthening Multi-doc QA (ASM QA). Following these tasks, our model excels in focusing more precisely on the desired information. Experimental results show substantial improvement in Multi-doc QA and other benchmarks, superior to state-of-the-art models by 13.7% absolute gain in shuffled settings, by 21.5% in passage retrieval task. We release our model, Ziya-Reader to promote related research in the community.

Relevant? yes / no

1

S33. TRUSTMEM: Learning Trustworthy Memory Consolidation for LLM Agents with Long-Term Memory (2026) Large language model (LLM) agents rely on long-term memory to support extended interactions and personalized assistance beyond finite context windows. Existing memory agents actively update external memory through generated write, revise, and delete operations, but these updates may omit important information, corrupt existing memory, or introduce unsupported hallucinated content. Once stored, such errors become persistent system-state failures that can affect future reasoning and generation. In this paper, we propose TrustMem, a framework designed to improve the trustworthiness of memory consolidation. TrustMem relies on a Memory Transition Verifier to evaluate the transition process of memory updates in terms of coverage, preservation, and faithfulness. It further constructs preference pairs among candidate updates under the same memory state, enabling preference-guided reinforcement learning to directly optimize memory updating behaviors. Extensive experiments demonstrate that TrustMem improves both memory utility and reliability: it achieves state-of-the-art results across MemoryAgentBench, HaluMem, and the Mem-alpha validation set, improves HaluMem memory extraction by 12.14 F1 points, and reduces transition-level omission, corruption, and hallucination by 40.1\{}%, 79.1\{}%, and 50.0\{}%, respectively, compared with the strongest baseline for each error type.

Relevant? yes / no

1

S34. The Retriever Should Remember: Experience-Amortized Reranking for Long-Term Agent Memory (2026) Long-term language-model agents accumulate memories across interactions, but their retrievers typically do not accumulate retrieval experience. Semantic retrieval is efficient, but embedding similarity does not always reflect whether a memory contains evidence relevant to the current query. Large language model (LLM) rerankers provide stronger query-conditioned relevance scores, yet stateless reranking repeatedly scores a large candidate pool and discards these scores after each query. We introduce EARM, an experience-amortized reranking framework that treats previously acquired LLM relevance scores as reusable retrieval experience. EARM stores sparse query–memory relevance scores in an online matrix, learns their shared structure through causal matrix completion, and combines a small set of newly observed scores with estimated scores to rerank the remaining candidates. The scoring budget decreases as experience accumulates, changing LLM reranking from a repeated per-query expense into a retrieval capability learned over an agent's lifetime. Experiments on long-term conversational memory show that mixed observed-and-estimated reranking improves answer accuracy over semantic retrieval by up to 6.62% and remains effective when only 17.5% of candidates receive direct LLM relevance scores, thereby substantially reducing the inference overhead of LLM reranking. These results motivate a broader view of agent memory: a long-lived agent should remember not only past content, but als

Relevant? yes / no

1

S35. ContextPilot: Teaching Agents for Proactive Context Management via Fine-grained RL (2026) Long-horizon agentic tasks require large language models (LLMs) to iteratively retrieve, integrate, and maintain dispersed information across multi-turn interactions, but preserving all interaction histories leads to a continuously growing working context. Recent proactive context management methods allow models to edit their own working context with specialized tools, yet they still face three key limitations: (1) a limited toolset restricted to search, deletion, and summarization, with no support for global planning, long-term memory, and adaptive compression; (2) inefficient exploration that treats context management actions uniformly despite their heterogeneous impacts

on final outcomes; and (3) coarse-grained credit assignment that assigns the final trajectory-level reward to all intermediate context editing actions during RL. To bridge these gaps, we introduce ContextPilot, a proactive context management framework for long-horizon agentic reasoning. Our approach systematically augments the toolset with planning, long-term memory, and soft context offloading tools. We further propose an RL method tailored for context management, which uses context and entropy variation to identify critical editing decisions for branch sampling and estimates action-level advantages from all branched trajectories that pass through the corresponding context editing action. Experiments on long-context QA and deep search tasks show that ContextPilot achieves stronger performance with a more c

Relevant? yes / no

1

S36. FINCH: Prompt-guided Key-Value Cache Compression for Large Language Models (2024) Abstract Recent large language model applications, such as Retrieval-Augmented Generation and chatbots, have led to an increased need to process longer input contexts. However, this requirement is hampered by inherent limitations. Architecturally, models are constrained by a context window defined during training. Additionally, processing extensive texts requires substantial GPU memory. We propose a novel approach, Finch, to compress the input context by leveraging the pre-trained model weights of the self-attention. Given a prompt and a long text, Finch iteratively identifies the most relevant Key (K) and Value (V) pairs over chunks of the text conditioned on the prompt. Only such pairs are stored in the KV cache, which, within the space constrained by the context window, ultimately contains a compressed version of the long text. Our proposal enables models to consume large inputs even with high compression (up to 93x) while preserving semantic integrity without the need for fine-tuning.

Relevant? yes / no

1

S37. EVICPRESS: Joint KV-Cache Compression and Eviction for Efficient LLM Serving (2025) Reusing KV cache is essential for high efficiency of Large Language Model (LLM) inference systems. With more LLM users, the KV cache footprint can easily exceed GPU memory capacity, so prior work has proposed to either evict KV cache to lower-tier storage devices, or compress KV cache so that more KV cache can be fit in the fast memory. However, prior work misses an important opportunity: jointly optimizing the eviction and compression decisions across all KV caches to minimize average generation latency without hurting quality. We propose EVICPRESS, a KV-cache management system that applies lossy compression and adaptive eviction to KV cache across multiple storage tiers. Specifically, for each KV cache of a context, EVICPRESS considers the effect of compression and eviction of the KV cache on the average generation quality and delay across all contexts as a whole. To achieve this, EVICPRESS proposes a unified utility function that quantifies the effect of quality and delay of the lossy compression or eviction. To this end, EVICPRESS's profiling module periodically updates the utility function scores on all possible eviction-compression configurations for all contexts and places KV caches using a fast heuristic to rearrange KV caches on all storage tiers, with the goal of maximizing the utility function scores on each storage tier. Compared to the baselines that evict KV cache or compress KV cache, EVICPRESS achieves higher KV-cache hit rates on fast devices, i.e., lower del

Relevant? yes / no

1

S38. Swin Transformer: Hierarchical Vision Transformer using Shifted Windows (2021) This paper presents a new vision Transformer, called Swin Transformer, that capably serves as a general-purpose backbone for computer vision. Challenges in adapting Transformer from language to vision arise from differences between the two domains, such as large variations in the scale of visual entities and the high resolution of pixels in images compared to words in text. To address these differences, we propose a hierarchical Transformer whose representation is computed with Shifted windows. The shifted windowing scheme brings greater efficiency by limiting self-attention computation to non-overlapping local windows while also allowing for cross-window connection. This hierarchical architecture has the flexibility to model at various scales and has linear computational complexity with respect to image size. These qualities of Swin Transformer make it compatible with a broad range of vision tasks, including image classification (87.3 top-1 accuracy on ImageNet-1K) and dense prediction tasks such as object detection (58.7 box AP and 51.1 mask AP on COCO test-dev) and semantic segmentation (53.5 mIoU on

ADE20K val). Its performance surpasses the previous state-of-the-art by a large margin of +2.7 box AP and +2.6 mask AP on COCO, and +3.2 mIoU on ADE20K, demonstrating the potential of Transformer-based models as vision backbones. The hierarchical design and the shifted window approach also prove beneficial for all-MLP architectures. The code and models are publicly available a

Relevant? yes / no

1

S39. Pruning Large Language Models to Intra-module Low-rank Architecture with Transitional Activations (2024) Structured pruning fundamentally reduces computational and memory overheads of large language models (LLMs) and offers a feasible solution for end-side LLM deployment. Structurally pruned models remain dense and high-precision, highly compatible with further tuning and compression. However, as the coarse-grained structured pruning poses large damage to the highly interconnected model, achieving a high compression ratio for scaled-up LLMs remains a challenge. In this paper, we introduce a task-agnostic structured pruning approach coupled with a compact Transformer architecture design. The proposed approach, named TransAct, reduces transitional activations inside multi-head attention (MHA) and multi-layer perceptron (MLP) modules, while preserving the inter-module activations that are sensitive to perturbations. Hence, the LLM is pruned into an intra-module low-rank architecture, significantly reducing weights, KV Cache and attention computation. TransAct is implemented on the LLaMA model and evaluated on downstream benchmarks. Results verify the optimality of our approach at high compression with respect to both efficiency and performance. Further, ablation studies reveal the strength of activation-guided iterative pruning and provide experimental analysis on the redundancy of MHA and MLP modules.

Relevant? yes / no

1

S40. Zero-Shot Instance Segmentation (2021) Deep learning has significantly improved the precision of instance segmentation with abundant labeled data. However, in many areas like medical and manufacturing, collecting sufficient data is extremely hard and labeling this data requires high professional skills. We follow this motivation and propose a new task set named zero-shot instance segmentation (ZSI). In the training phase of ZSI, the model is trained with seen data, while in the testing phase, it is used to segment all seen and unseen instances. We first formulate the ZSI task and propose a method to tackle the challenge, which consists of Zero-shot Detector, Semantic Mask Head, Background Aware RPN and Synchronized Background Strategy. We present a new benchmark for zero-shot instance segmentation based on the MS-COCO dataset. The extensive empirical results in this benchmark show that our method not only surpasses the state-of-the-art results in zero-shot object detection task but also achieves promising performance on ZSI. Our approach will serve as a solid baseline and facilitate future research in zero-shot instance segmentation. Code available at $\{}color{Magenta}{\{}text{ZSI}}$.

Relevant? yes / no